



10-x23 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Vision-Language Models (VLMs)

Matt Gormley & Aran Nayebi

Lecture 13

Reminders

- **Homework 3: Applying and Adapting LLMs**
 - Out: Sat, Feb 21
 - Due: Thu, Mar 12 at 11:59pm

VISION LANGUAGE MODELS (VLMS)

Multimodal Models

- Previously: Text-to-image models – adapt generative models for vision in order to guide their output toward some desired target using natural language
 - Output is still an image
- Today: visual language models (VLMs) – adapt generative models for text in order to allow them to interact with images (as well as text) as input
 - Output is (typically) still text

VLM: Tasks

- Common benchmarks for VLMs include
 - **Visual reasoning:** given an image (or a pair of images) determine if some natural language statement about the image(s) is true or false
 - **Visual grounding:** locate an object in some image given a natural language description
 - **Visual question answering:** given an image (or images), respond to arbitrary, potentially open-ended questions about the content.
 - **Caption generation:** create natural language descriptions of content of some image

VLM: Tasks

- Common benchmarks for VLMs include
 - **Visual reasoning:** given an image (or a pair of images) determine if some natural language statement about the image(s) is true or false



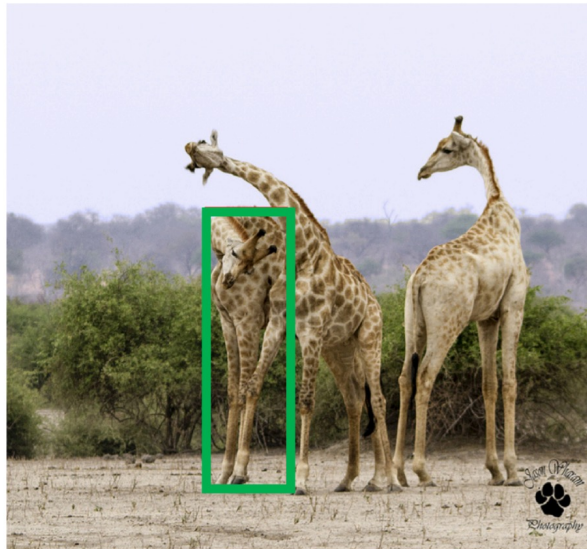
The left image contains twice the number of dogs as the right image, and at least two dogs in total are standing.



One image shows exactly two brown acorns in back-to-back caps on green foliage.

VLM: Tasks

- Common benchmarks for VLMs include
 - **Visual reasoning:** given an image (or a pair of images) determine if some natural language statement about the image(s) is true or false
 - **Visual grounding:** locate an object in some image given a natural language description



RefCOCO:

1. giraffe on left
2. first giraffe on left

RefCOCO+:

1. giraffe with lowered head
2. giraffe head down

RefCOCog:

1. an adult giraffe scratching its back with its horn
2. giraffe hugging another giraffe

VLM: Tasks

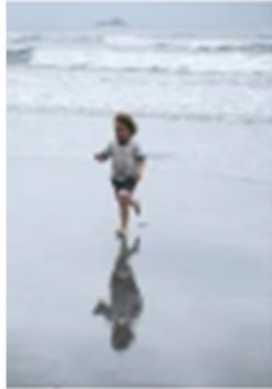
- Common benchmarks for VLMs include



- Visual question answering:** given an image (or images), respond to arbitrary, potentially open-ended questions about the content.

VLM: Tasks

- Common benchmarks for VLMs include



Ground Truth Caption: A little boy runs away from the approaching waves of the ocean.

Generated Caption: A young boy is running on the beach.



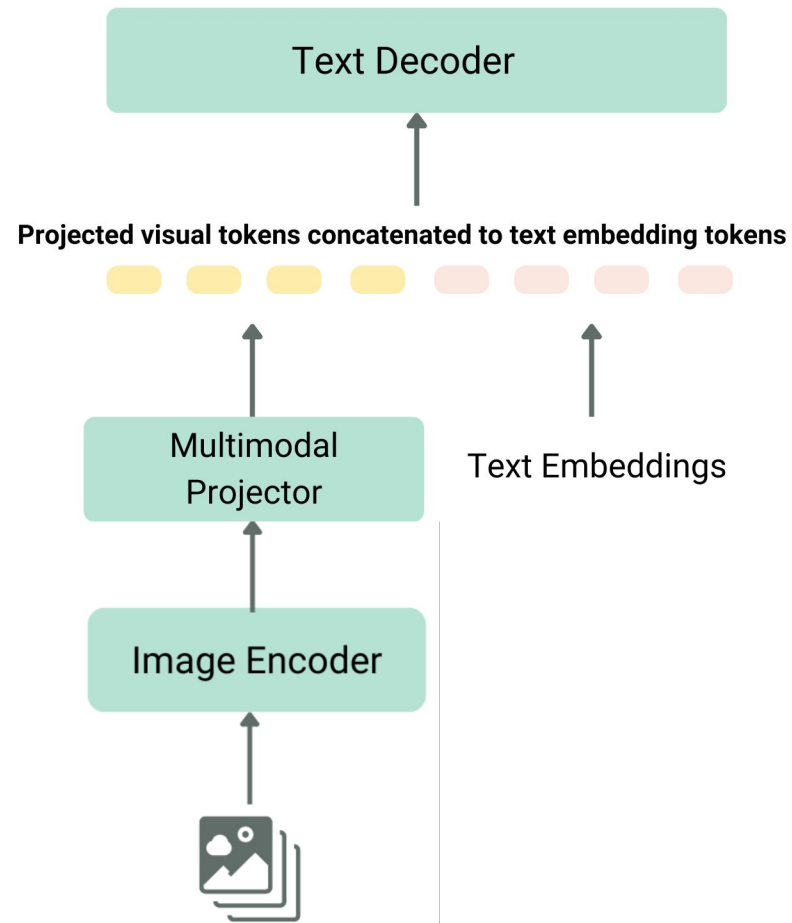
Ground Truth Caption: A brunette girl wearing sunglasses and a yellow shirt.

Generated Caption: A woman in a black shirt and sunglasses smiles.

- **Caption generation:** create natural language descriptions of content of some image

VLM: Architecture

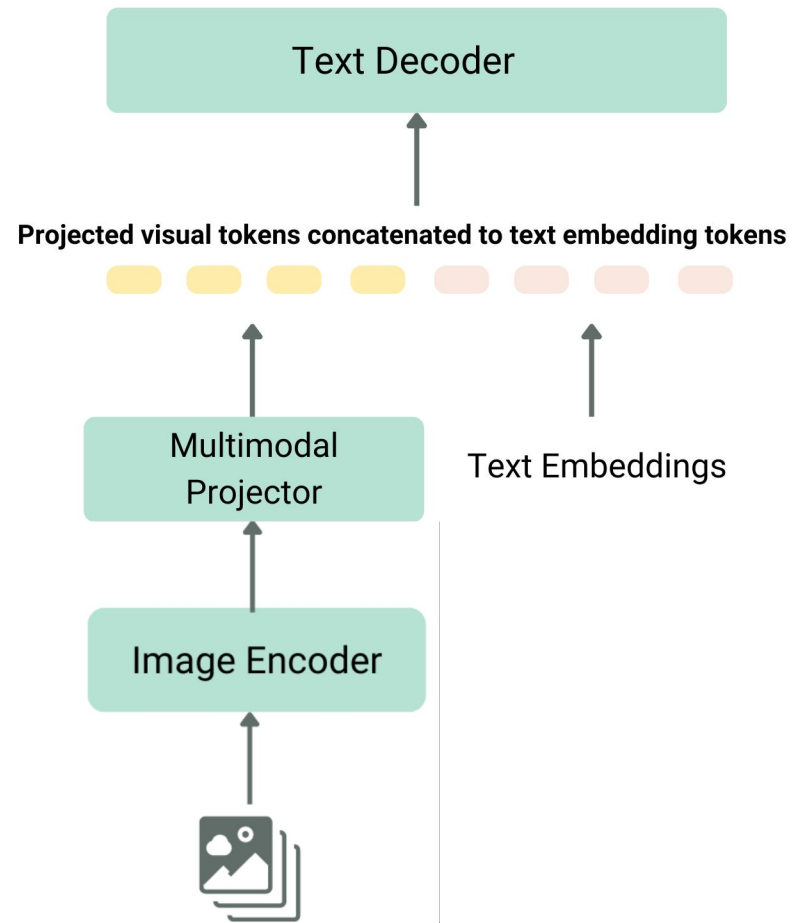
- High-level idea: convert both the image and the text inputs into embedding vectors, then pass those vectors into a decoder-only transformer and do next (text) token prediction



- Two common encoders:
 - VQ-VAE encoder followed by an embedding layer that converts the discrete tokens into dense numerical vectors
 - CLIP encoder, that directly learns an embedding vector using a contrastive pre-training objective

VLM: Architecture

- High-level idea: convert both the image and the text inputs into embedding vectors, then pass those vectors into a decoder-only transformer and do next (text) token prediction



- Two common encoders:
 - VQ-VAE encoder followed by an embedding layer that converts the discrete tokens into dense numerical vectors
 - CLIP encoder, that directly learns an embedding vector using a contrastive pre-training objective

IMAGE ENCODERS

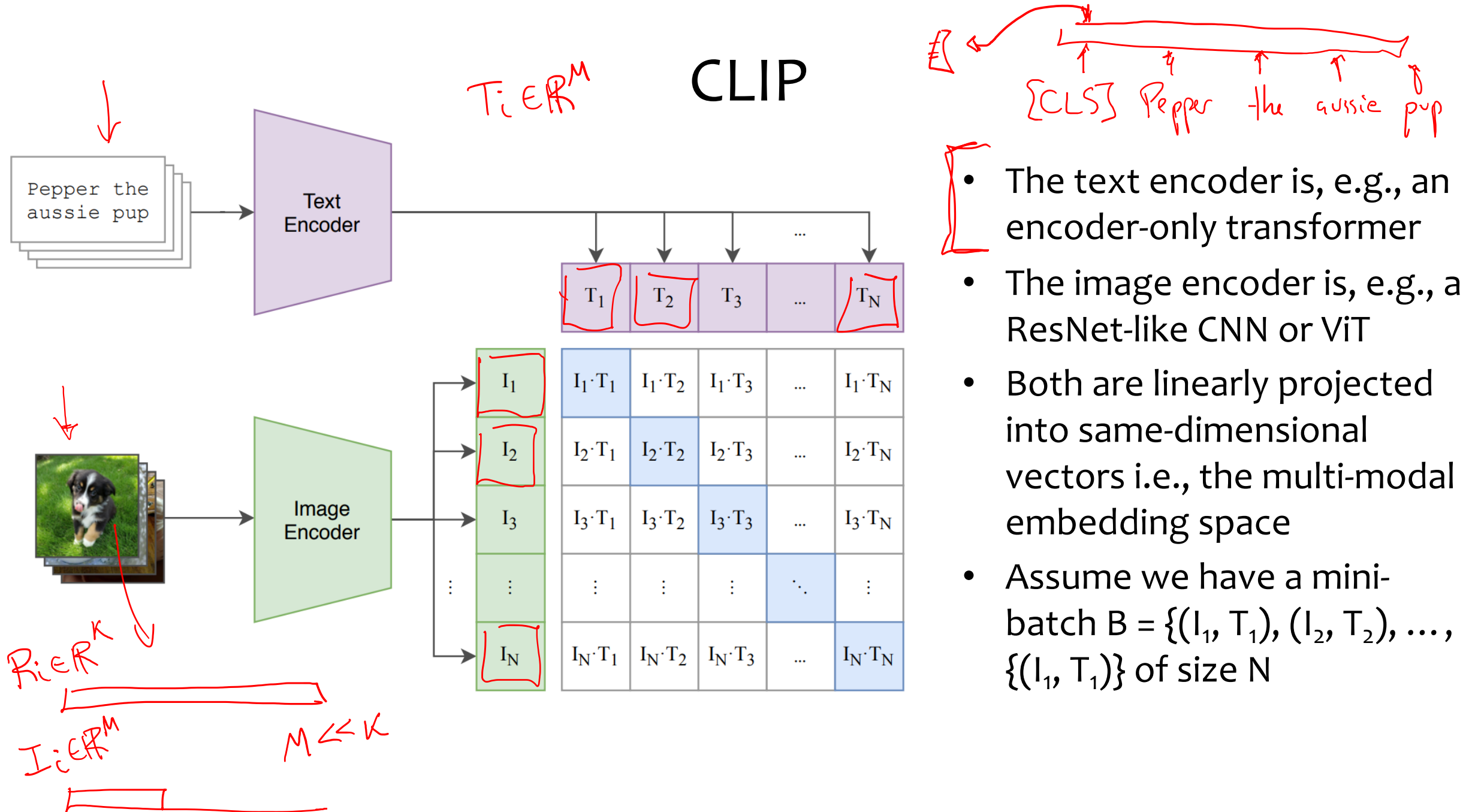
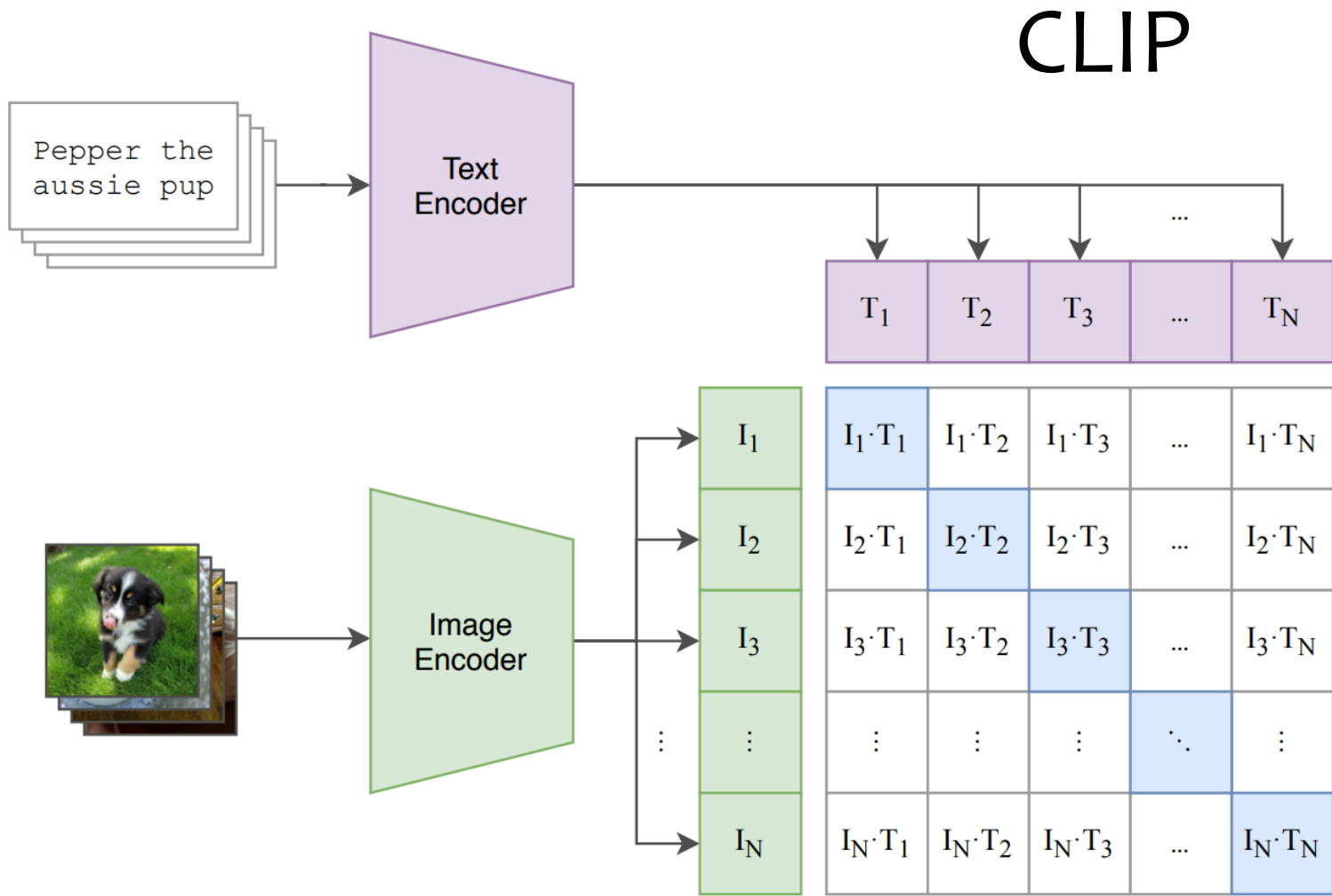


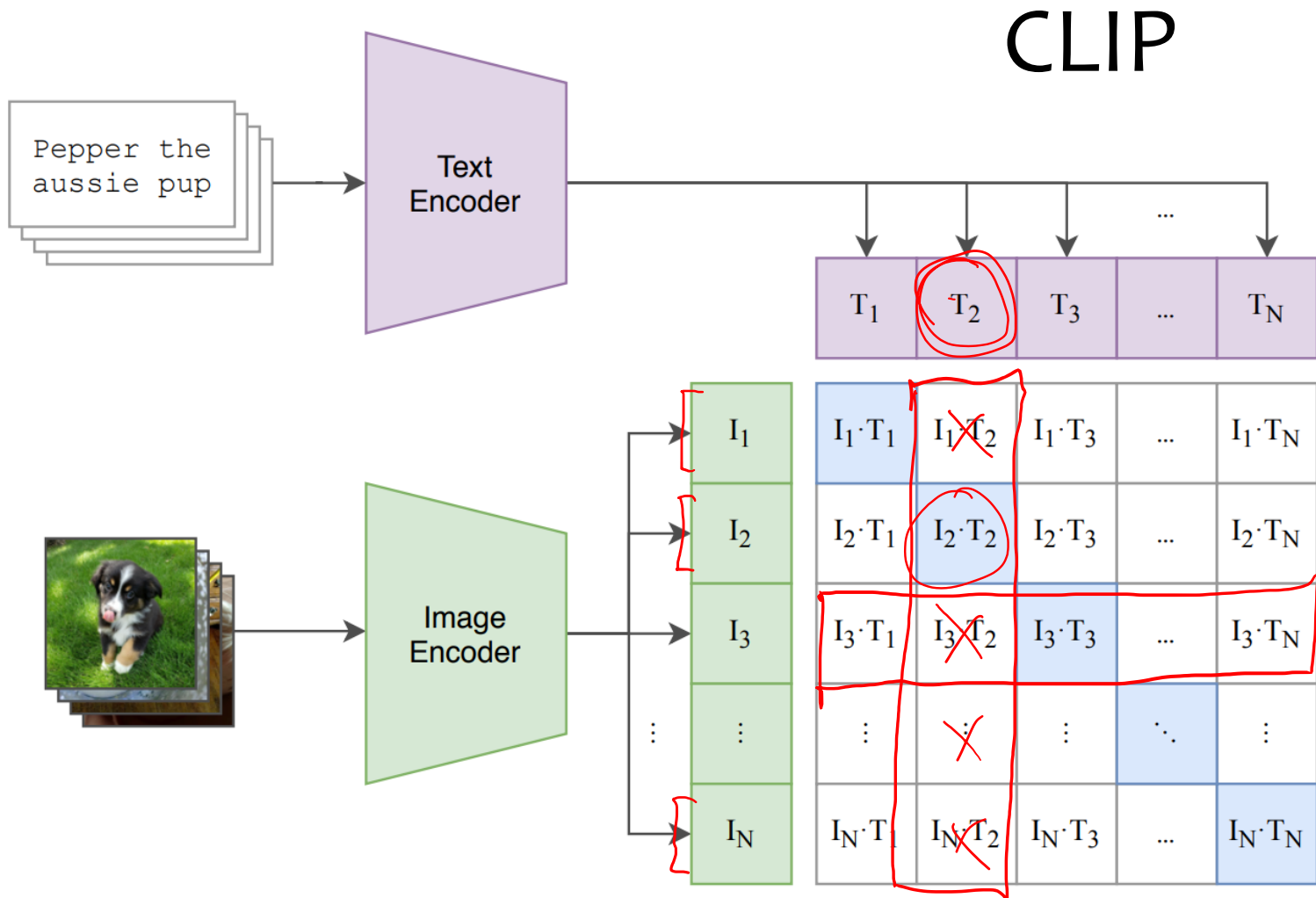
Figure from Radford et al. (2021) <https://arxiv.org/pdf/2103.00020>



**Incorrect (but intuitive)
objective function:**

$$\max \left[\sum_{i=1}^N I_i^\top T_i - \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N I_i^\top T_j \right]$$

Given a mini-batch of N (image, caption) pairs, both encoders are simultaneously pre-trained to maximize the cosine similarity of corresponding image-caption embedding vectors and minimize all other pairwise cosine similarities



Correct objective function:

$$\max \sum_{i=1}^N \left[\log \frac{\exp \left(\frac{I_i^\top T_i}{\tau} \right)}{\sum_{j=1}^N \exp \left(\frac{I_i^\top T_j}{\tau} \right)} + \log \frac{\exp \left(\frac{I_i^\top T_i}{\tau} \right)}{\sum_{j=1}^N \exp \left(\frac{I_j^\top T_i}{\tau} \right)} \right]$$

Given a mini-batch of N (image, caption) pairs, both encoders are simultaneously pre-trained to maximize the cosine similarity of corresponding image-caption embedding vectors and minimize all other pairwise cosine similarities

CLIP

Correct objective function:

Can be interpreted in terms of two conditional probability distributions:

1. Image-to-Text distribution:

$$p_{IT}(i \text{ matches } j \mid I_i, T_1, \dots, T_N) = \frac{\exp\left(\frac{I_i^\top T_j}{\tau}\right)}{\sum_{k=1}^N \exp\left(\frac{I_i^\top T_k}{\tau}\right)}$$

(softmax over ~~columns~~ rows)

2. Text-to-Image distribution:

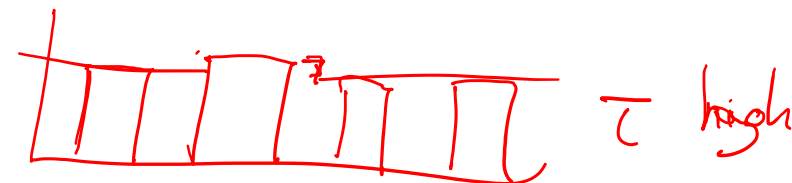
$$p_{TI}(i \text{ matches } j \mid T_i, I_1, \dots, I_N) = \frac{\exp\left(\frac{I_j^\top T_i}{\tau}\right)}{\sum_{k=1}^N \exp\left(\frac{I_k^\top T_i}{\tau}\right)}$$

(softmax over ~~rows~~ cols)

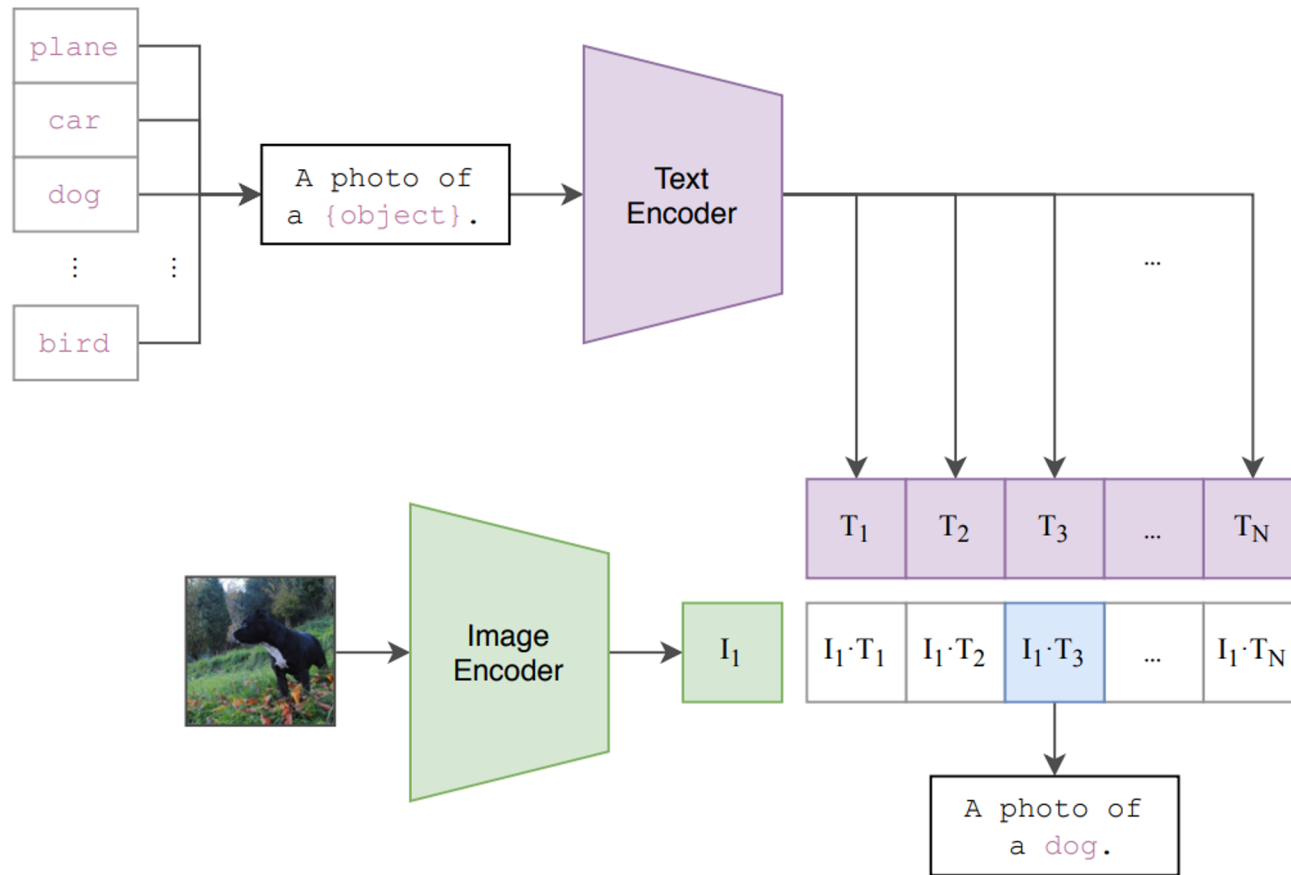
The bidirectional **contrastive loss** maximizes the log-likelihood of the correct (matched) pairs under both distributions:

$$\sum_{i=1}^N \log p_{IT}(i \text{ matches } i \mid I_i, T_1, \dots, T_N) + \log p_{TI}(i \text{ matches } i \mid T_i, I_1, \dots, I_N)$$

$$\max \sum_{i=1}^N \left[\log \frac{\exp\left(\frac{I_i^\top T_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{I_i^\top T_j}{\tau}\right)} + \log \frac{\exp\left(\frac{I_i^\top T_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{I_j^\top T_i}{\tau}\right)} \right]$$



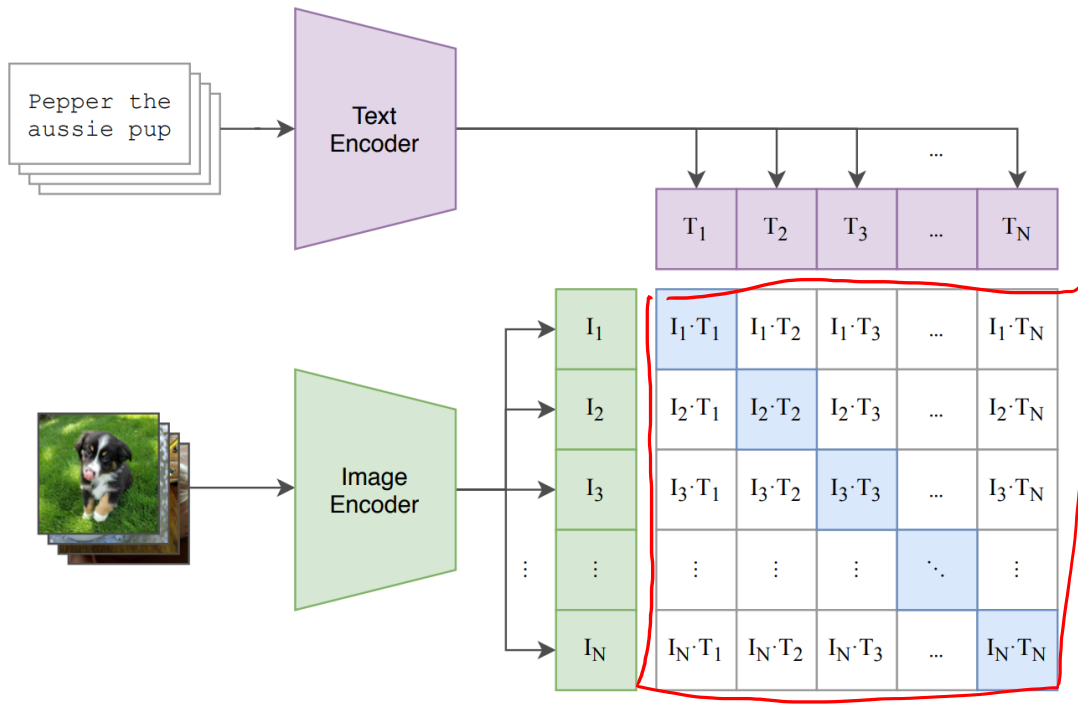
CLIP for Zero Shot Classification



Among the N labels, choose the one that has the highest probability:

$$\hat{y} = \operatorname{argmax}_i p_{\text{IT}}(i \text{ matches } 1 \mid I_1, T_1, \dots, T_N)$$

$$= \operatorname{argmax}_i \frac{\exp\left(\frac{I_1^\top T_i}{\tau}\right)}{\sum_{j=1}^N \exp\left(\frac{I_1^\top T_j}{\tau}\right)}$$



- The CLIP objective function works fine if your batch sizes are small enough to fit on a single machine
- But it creates communication overhead if you need to spread the above matrix across many machines
- SigLIP partly solves this issue by replacing the softmax with a sigmoid

SigLIP

CLIP objective function:

$$\sum_{i=1}^N \left[\log \frac{\exp \left(\frac{I_i^\top T_i}{\tau} \right)}{\sum_{j=1}^N \exp \left(\frac{I_i^\top T_j}{\tau} \right)} + \log \frac{\exp \left(\frac{I_i^\top T_i}{\tau} \right)}{\sum_{j=1}^N \exp \left(\frac{I_j^\top T_i}{\tau} \right)} \right]$$

SigLIP objective function:

$$\max \sum_{i=1}^N \sum_{j=1}^N \log \frac{1}{1 + \exp(y_{ij}(-w I_i^\top T_j + b))}$$

where $y_{ij} = \begin{cases} 1 & \text{if } i = j \text{ (matching pair)} \\ -1 & \text{otherwise.} \end{cases}$

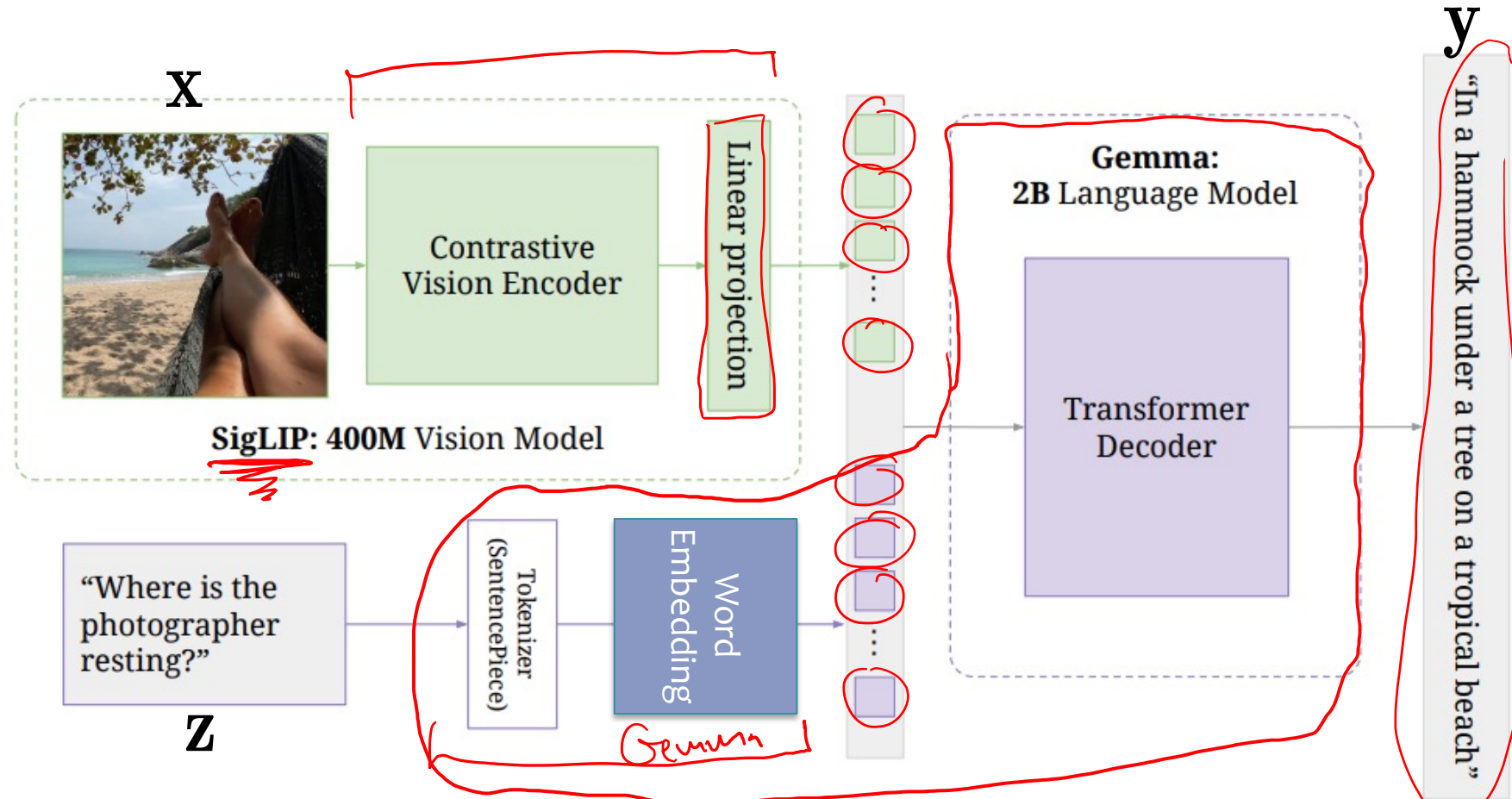
and $w = \exp(w')$ and b are learnable parameters

(VLMs that read text/images but only write text)

VLMS WITH TEXT-ONLY DECODERS

PaliGemma

- SigLIP is a variant of CLIP
- Gemma is a 2B LLM (open source counterpart to Gemini)
- Linear projection is creating image embeddings, but they live in the same high dimensional space as the word embeddings
- The LLM has to figure out how to use the image embeddings in some useful way so as to maximize the likelihood of the correct text response



$$\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{z}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$$

Qwen-VL

Freezing subsets of the model parameters (corresponding to pretrained submodels) ~~is~~ ^{was} common during training

Qwen-VL as an example:

1. Qwen-VL first freezes the LLM in order to learn effective image embeddings that align with the word space
2. Then, all parameters are unfrozen for a while
3. Finally, it freezes only the ViT image encoder

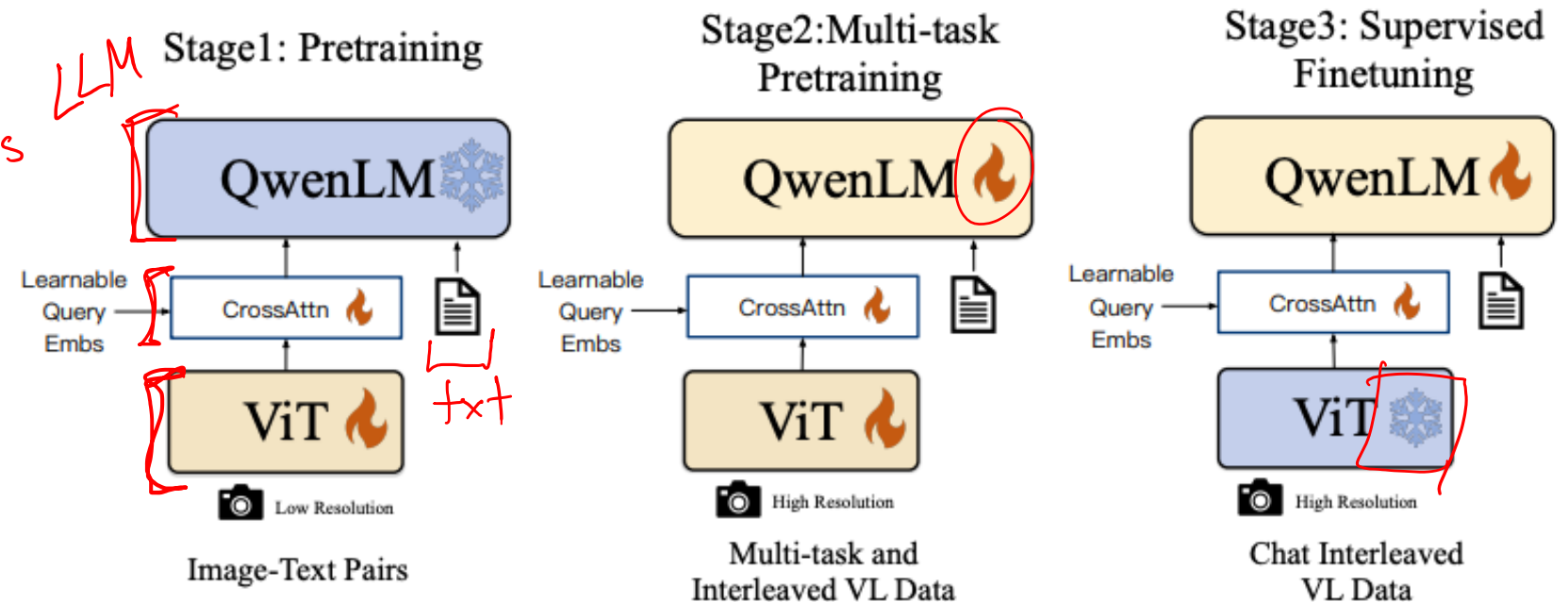
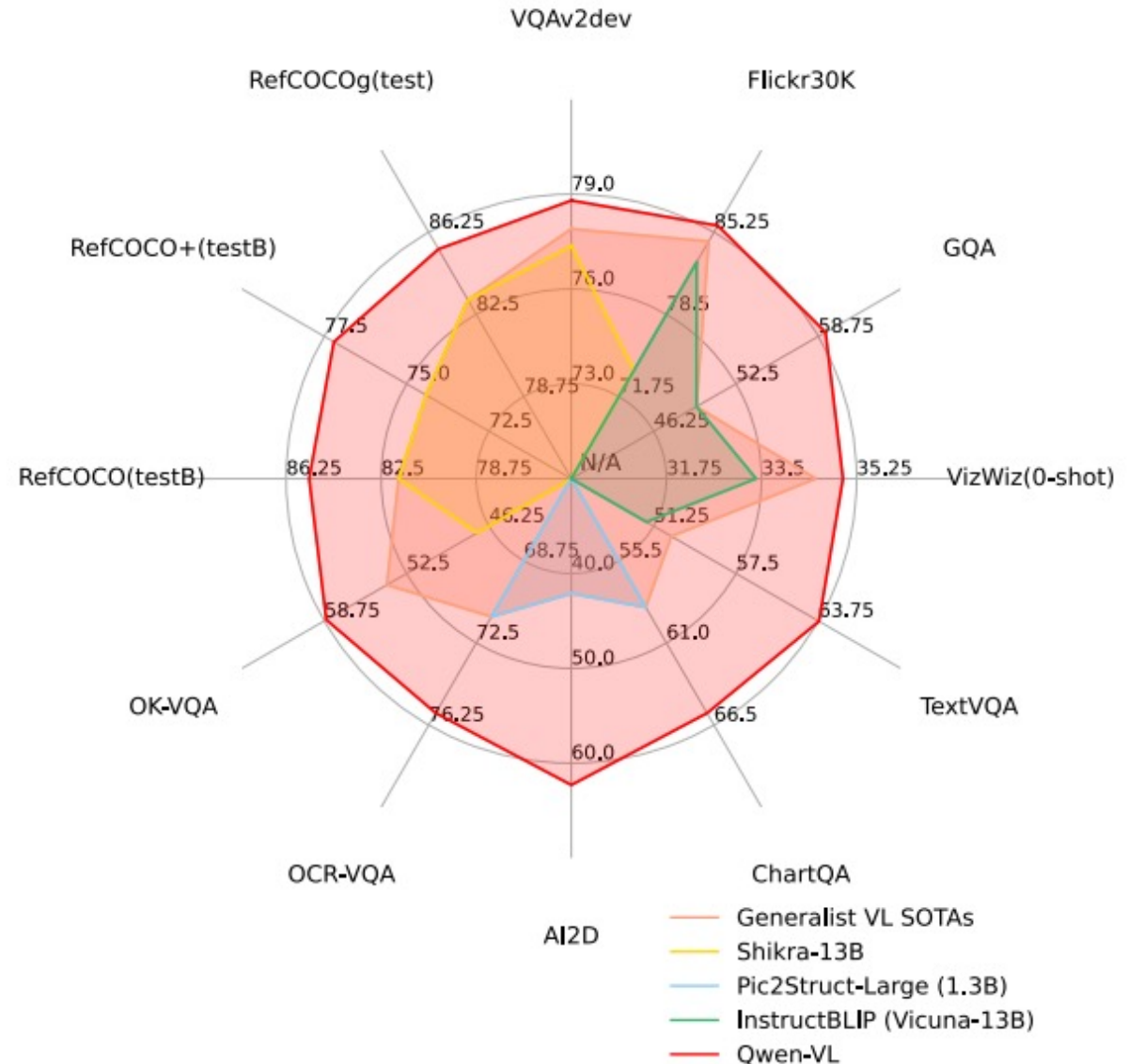


Figure 3: The training pipeline of the Qwen-VL series.

Qwen-VL

Just as LLMs are typically evaluated on a variety of text benchmarks assessing the understanding / generation abilities of the model...

...VLMs are evaluated on a variety of text/image benchmarks assessing their ability to interpret and respond to queries about multimodal data.



Llama 3.2 Vision

Just as LLMs are typically evaluated on a variety of text benchmarks assessing the understanding / generation abilities of the model...

...VLMs are evaluated on a variety of text/image benchmarks assessing their ability to interpret and respond to queries about multimodal data.

Vision instruction-tuned benchmarks

Modality	Category Benchmark	Llama 3.2 11B	Llama 3.2 90B	Claude 3 – Haiku	GPT-4o-mini
Image	College-level Problems and Mathematical Reasoning MMMU (val, 0-shot CoT, micro avg accuracy)	50.7	60.3	50.2	59.4
	MMMU-Pro, Standard (10 opts, test)	33.0	45.2	27.3	42.3
	MMMU-Pro, Vision (test)	23.7	33.8	20.1	36.5
	MathVista (testmini)	51.5	57.3	46.4	56.7
	Charts and Diagram Understanding ChartQA (test, 0-shot CoT relaxed accuracy)*	83.4	85.5	81.7	—
	AI2 Diagram (test)*	91.1	92.3	86.7	—
	DocVQA (test, ANLS)*	88.4	90.1	88.8	—
	General Visual Question Answering VQAv2 (test)	75.2	78.1	—	—
Text	General MMLU (0-shot, CoT)	73.0	86.0	75.2 (5-shot)	82.0
	Math MATH (0-shot, CoT)	51.9	68.0	38.9	70.2
	Reasoning GPQA (0-shot, CoT)	32.8	46.7	33.3	40.2
	Multilingual MGSM (0-shot, CoT)	68.9	86.9	75.1	87.0

VLMs and Image Resolution

Fixed Size Image Inputs

- Qwen-VL (the original model) requires **fixed size images** as inputs
- Each image is resized to 448x448
- Then the image is converted to a fixed length representation: 256 embeddings
- Combined with absolute positional embeddings

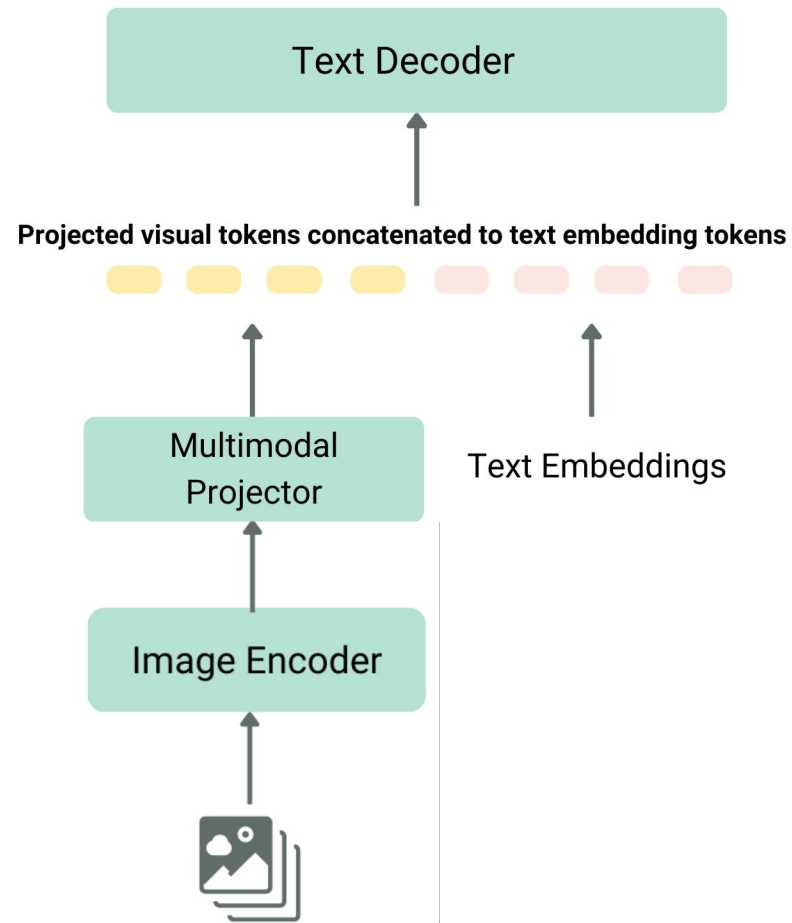
Dynamic Image Inputs

- Qwen2-VL supports **dynamic image resolution**
- ViT still works on patches of size 28x28, so dimensions are resized to be a multiple of 28
- The supported range of image sizes is: 56x56 to 3584x3585
- Modified ViT to use 2D-RoPE positional embeddings

VISION LANGUAGE MODELS (VLMS)

VLM: Architecture

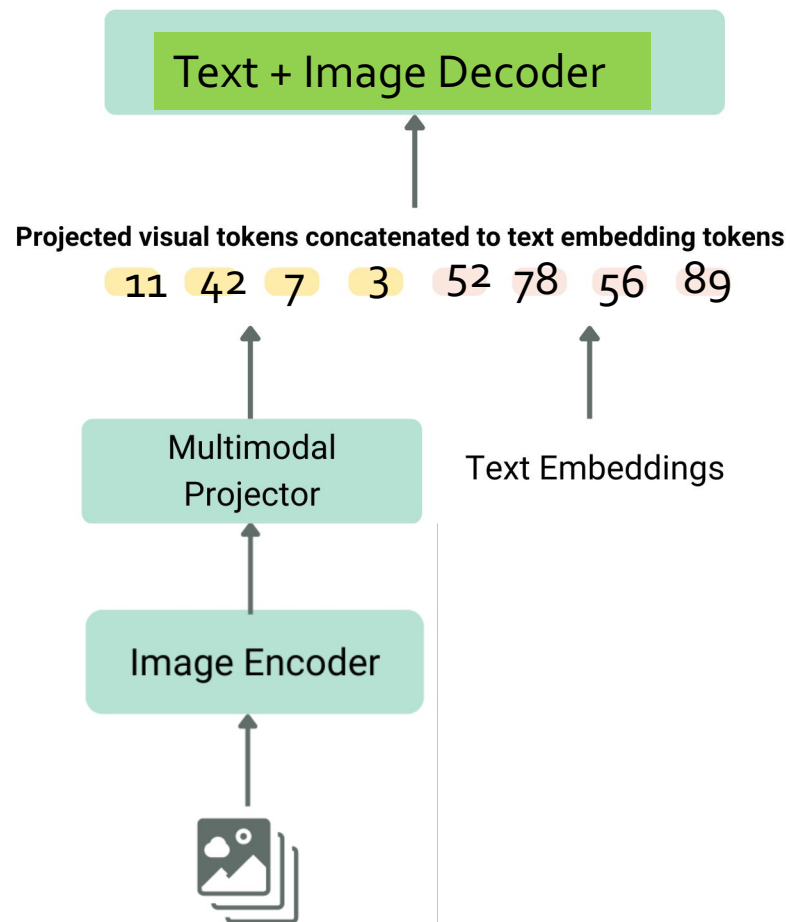
- High-level idea: convert both the image and the text inputs into embedding vectors, then pass those vectors into a decoder-only transformer and do next (text) token prediction



- Two common encoders:
 - VQ-VAE encoder followed by an embedding layer that converts the discrete tokens into dense numerical vectors
 - CLIP encoder, that directly learns an embedding vector using a contrastive pre-training objective

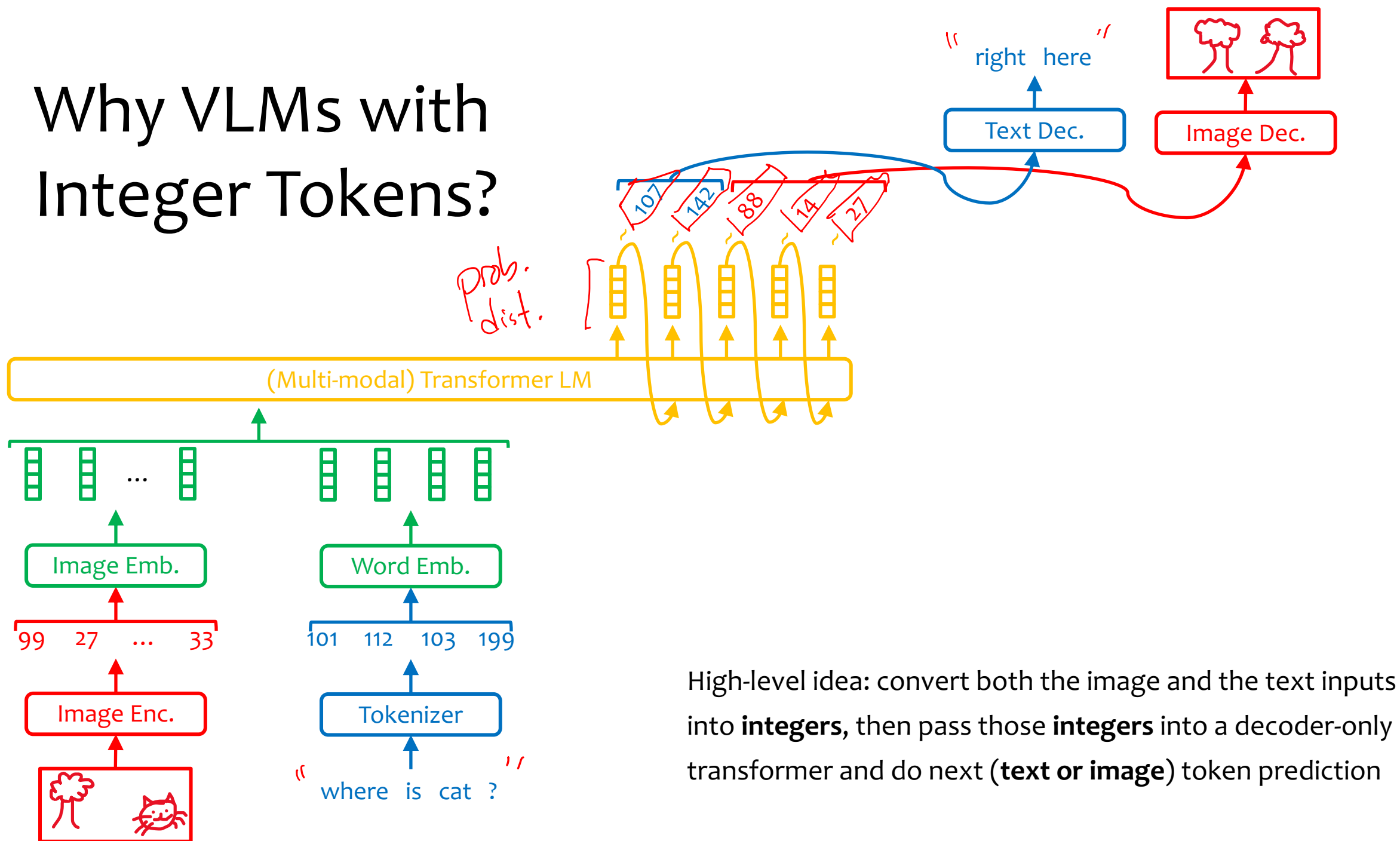
VLM: Architecture

- High-level idea: convert both the image and the text inputs into **integers**, then pass those **integers** into a decoder-only transformer and do next (**text or image**) token prediction



- Two common encoders:
 - VQ-VAE encoder followed by an embedding layer that converts the discrete tokens into dense numerical vectors
 - CLIP encoder, that directly learns an embedding vector using a contrastive pre-training objective

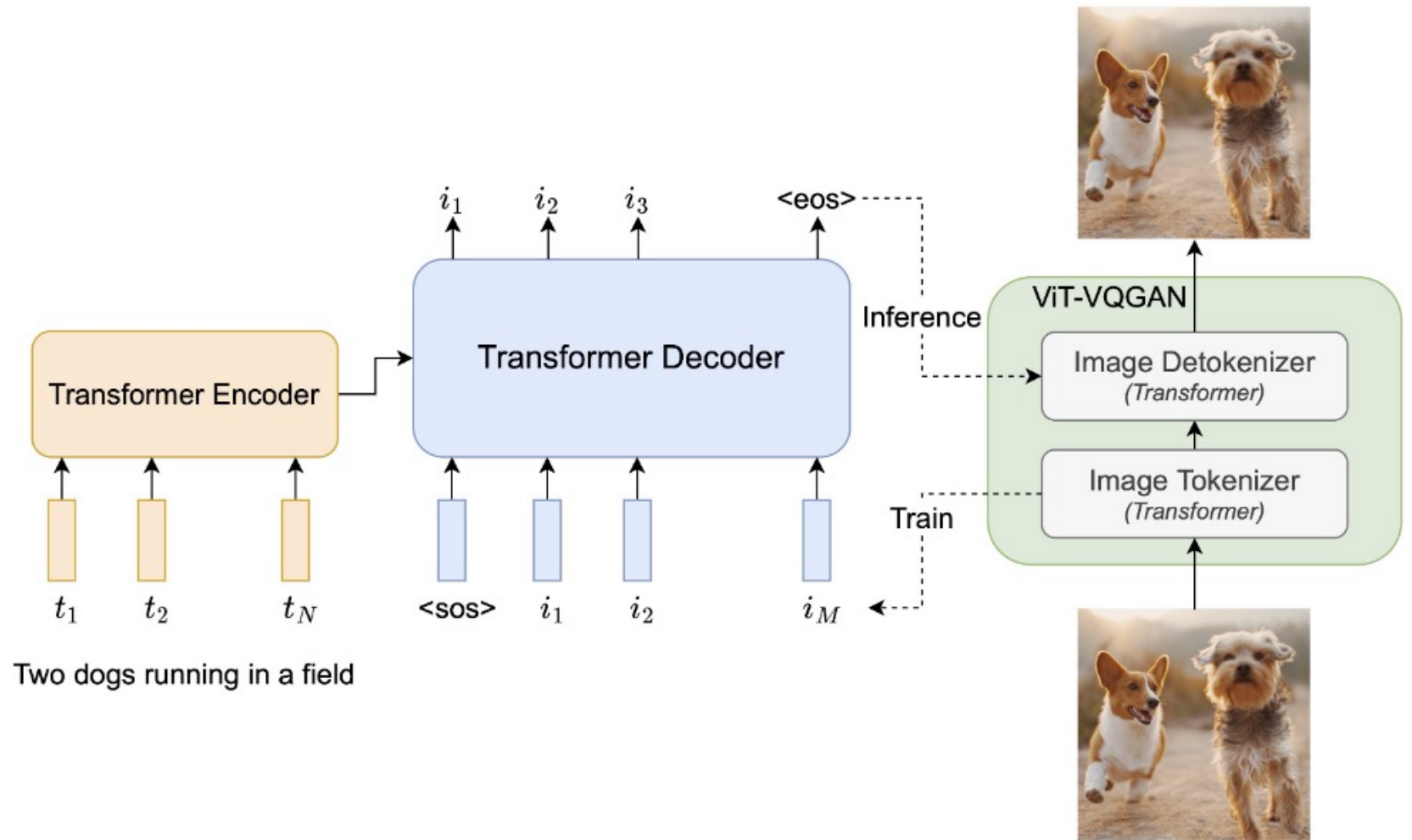
Why VLMs with Integer Tokens?



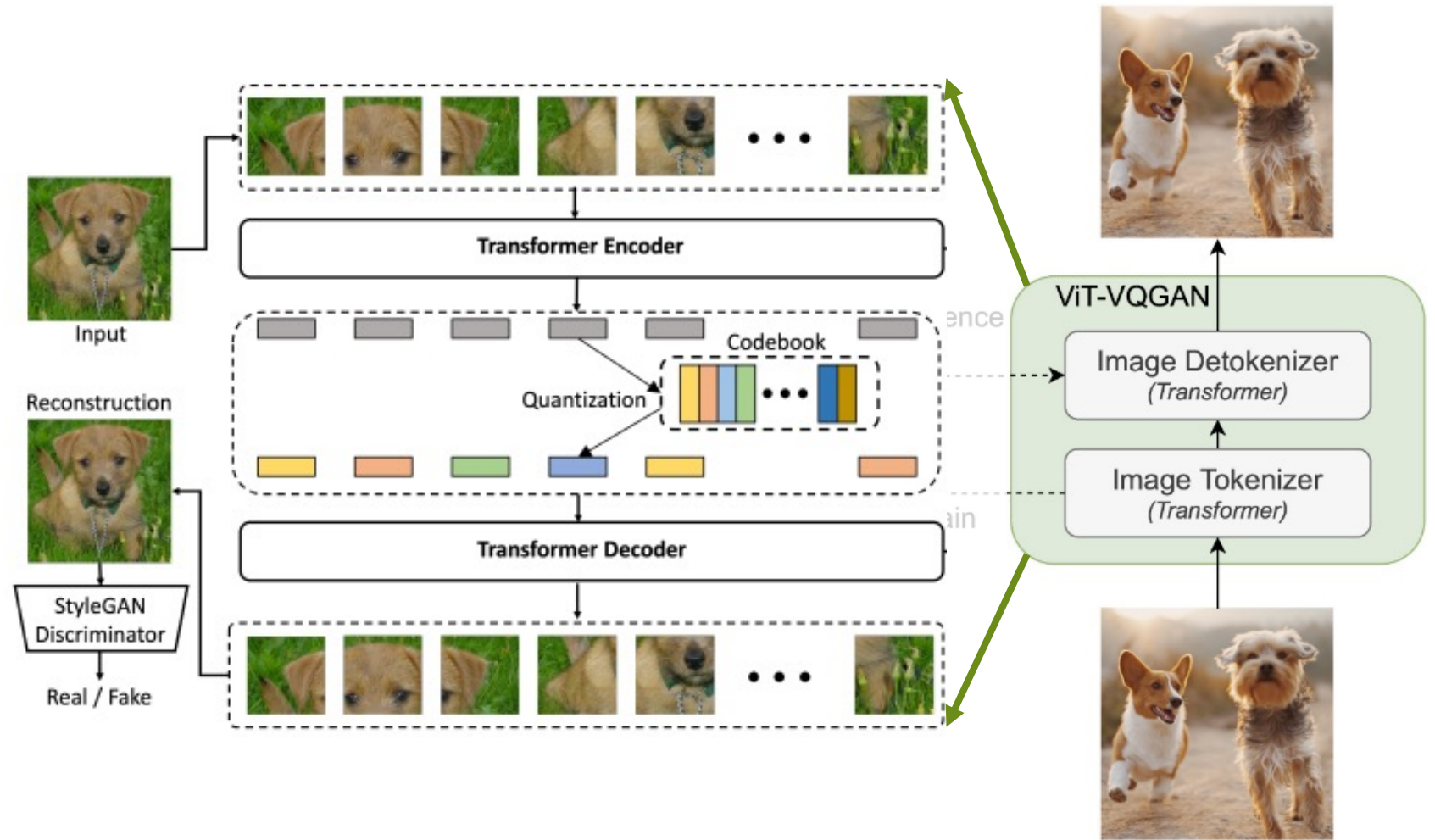
High-level idea: convert both the image and the text inputs into **integers**, then pass those **integers** into a decoder-only transformer and do next (**text or image**) token prediction

VQ-VAES

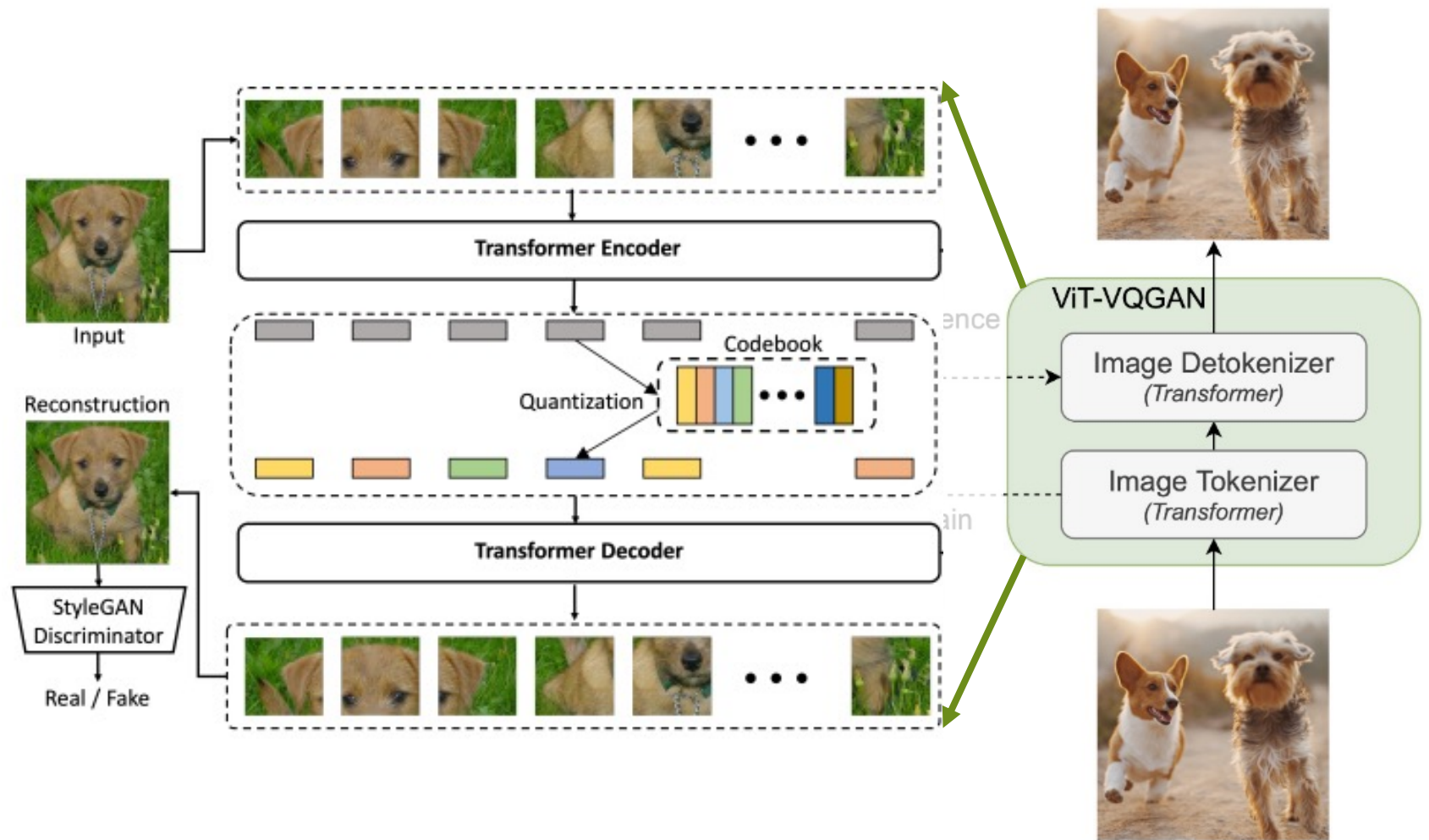
Recall: Parti

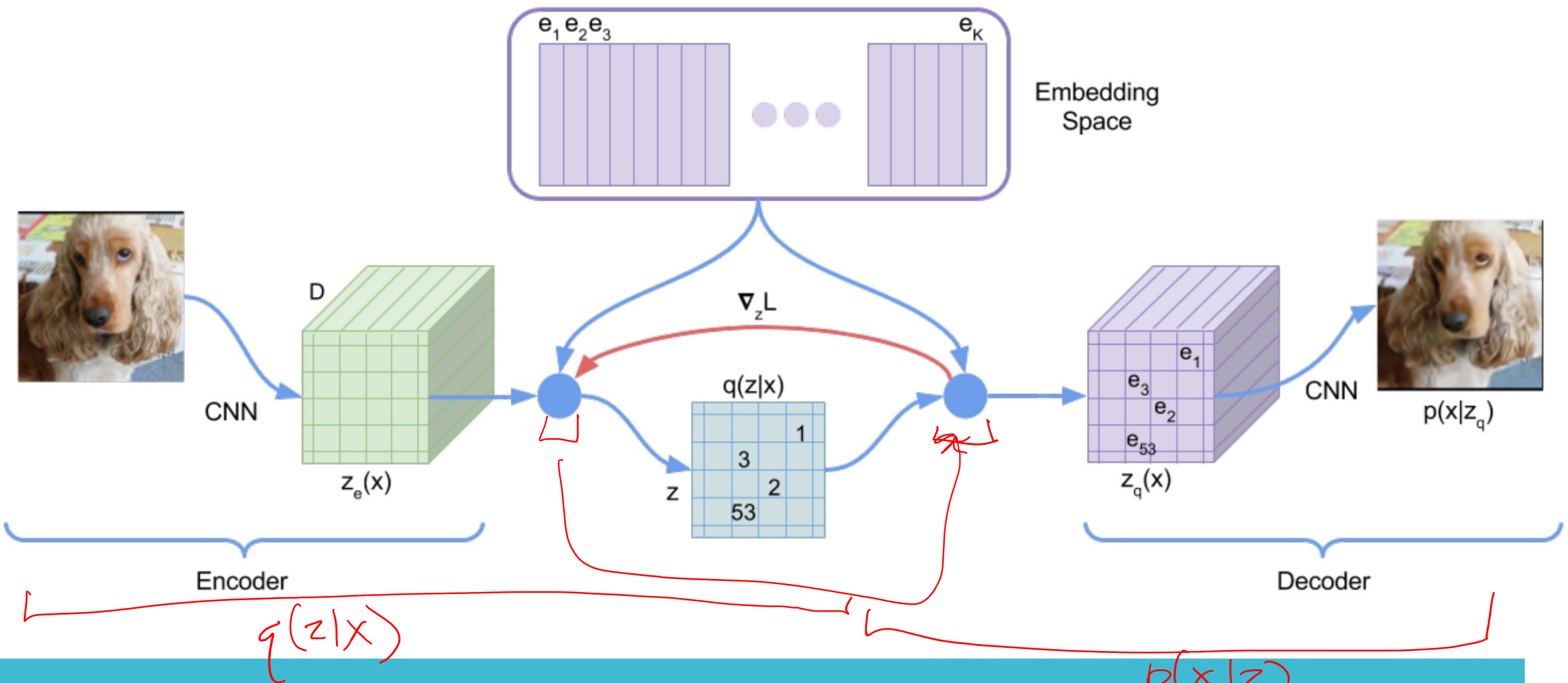


Recall: Image Tokenization

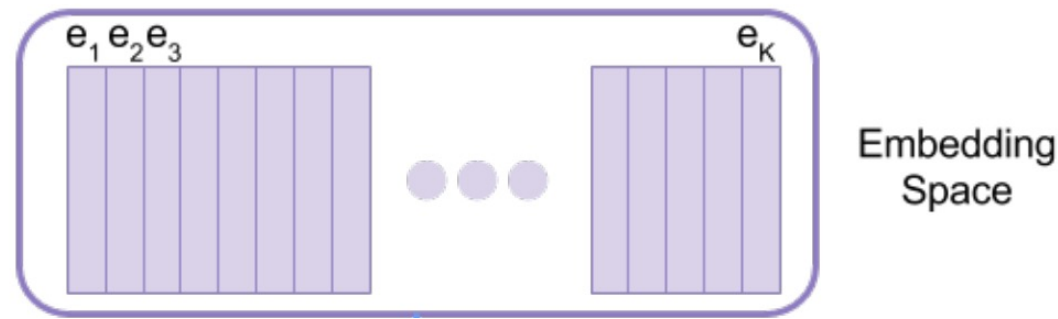


How can we
(pre-)train
these models
given the non-
differentiable
quantization
operation?





Vector-Quantized VAEs

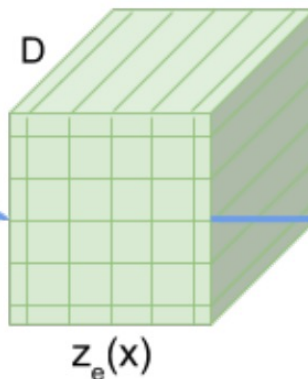


- Embedding space consists of K D -dimensional latent vectors $\{e_1, \dots, e_K\}$ which are learned during training
- The indices $[1, \dots, K]$ of each latent vector correspond to the “image tokens” in some fixed-length codebook

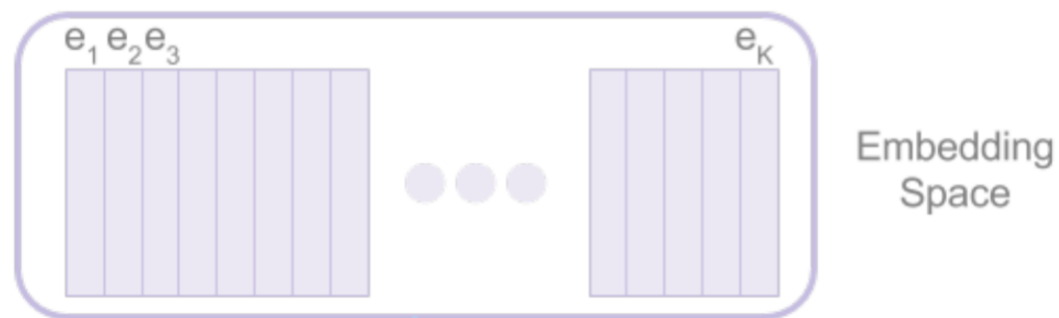
Vector-Quantized VAEs



CNN

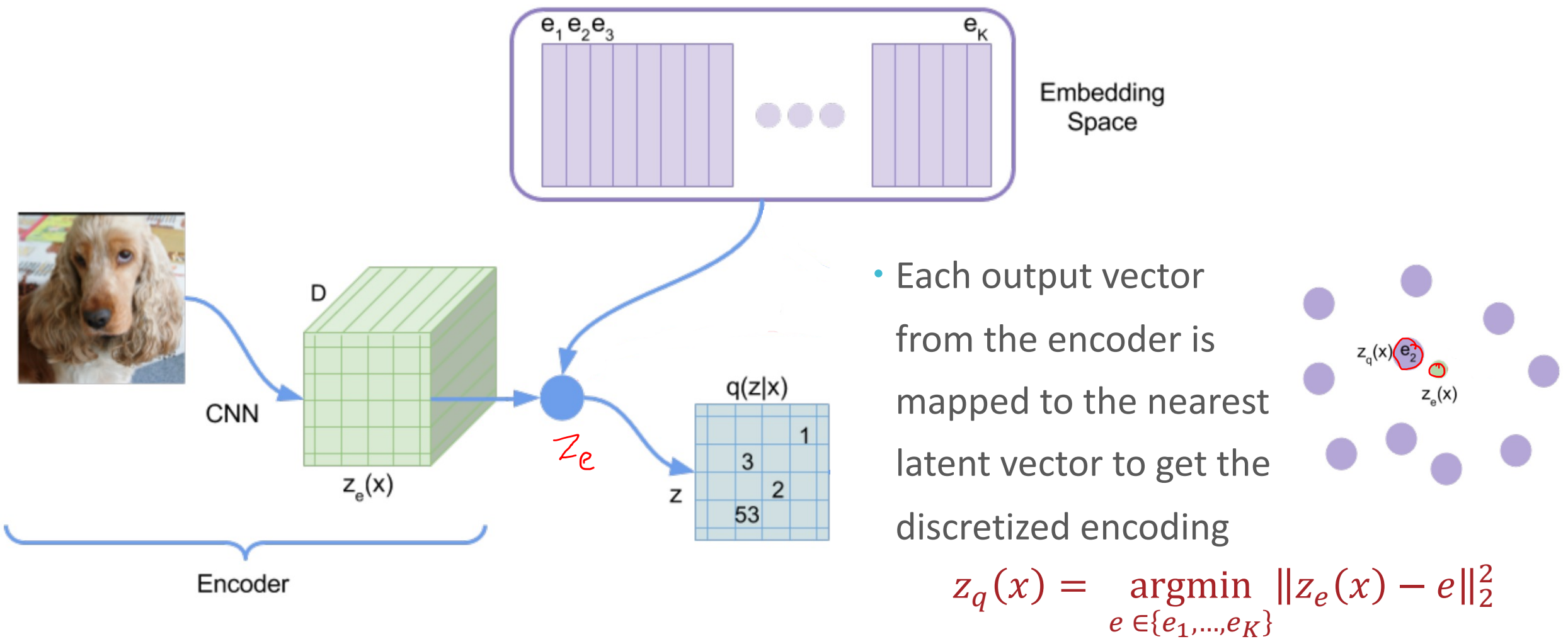


Encoder

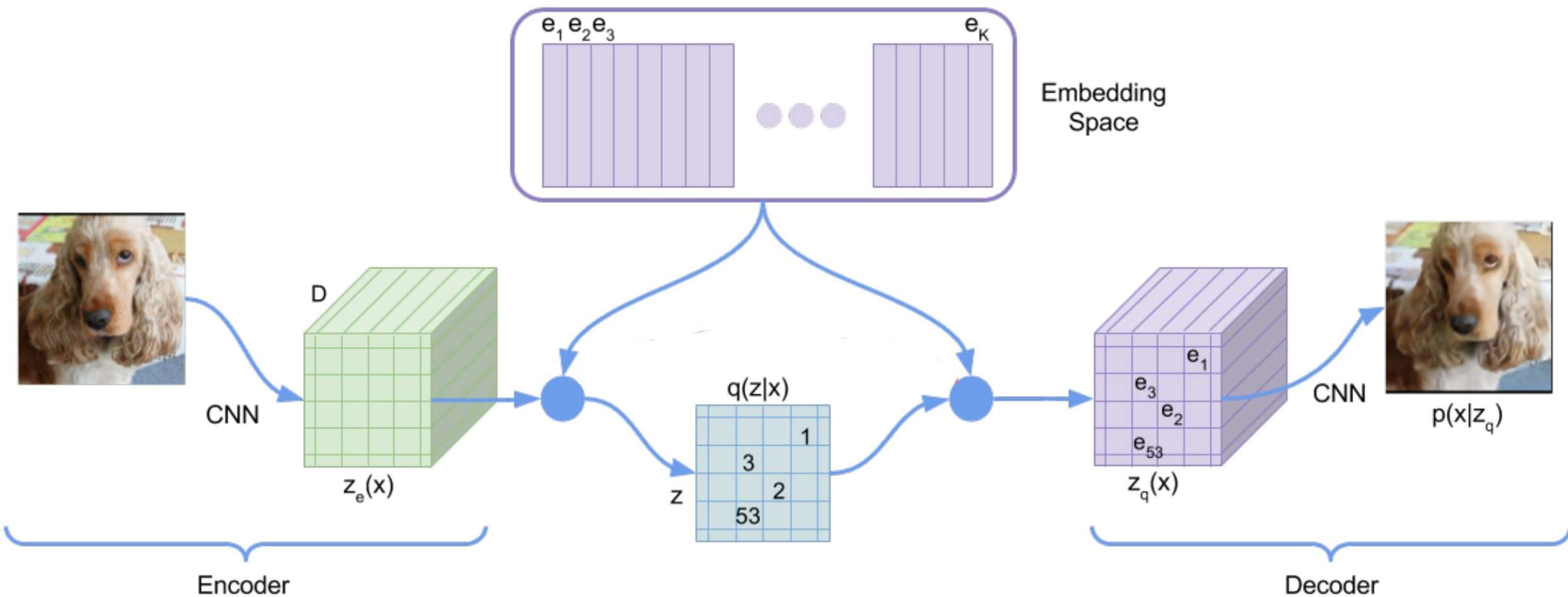


- The encoder (e.g., a ResNet-like CNN) maps images to N D -dimensional vectors

Vector-Quantized VAEs

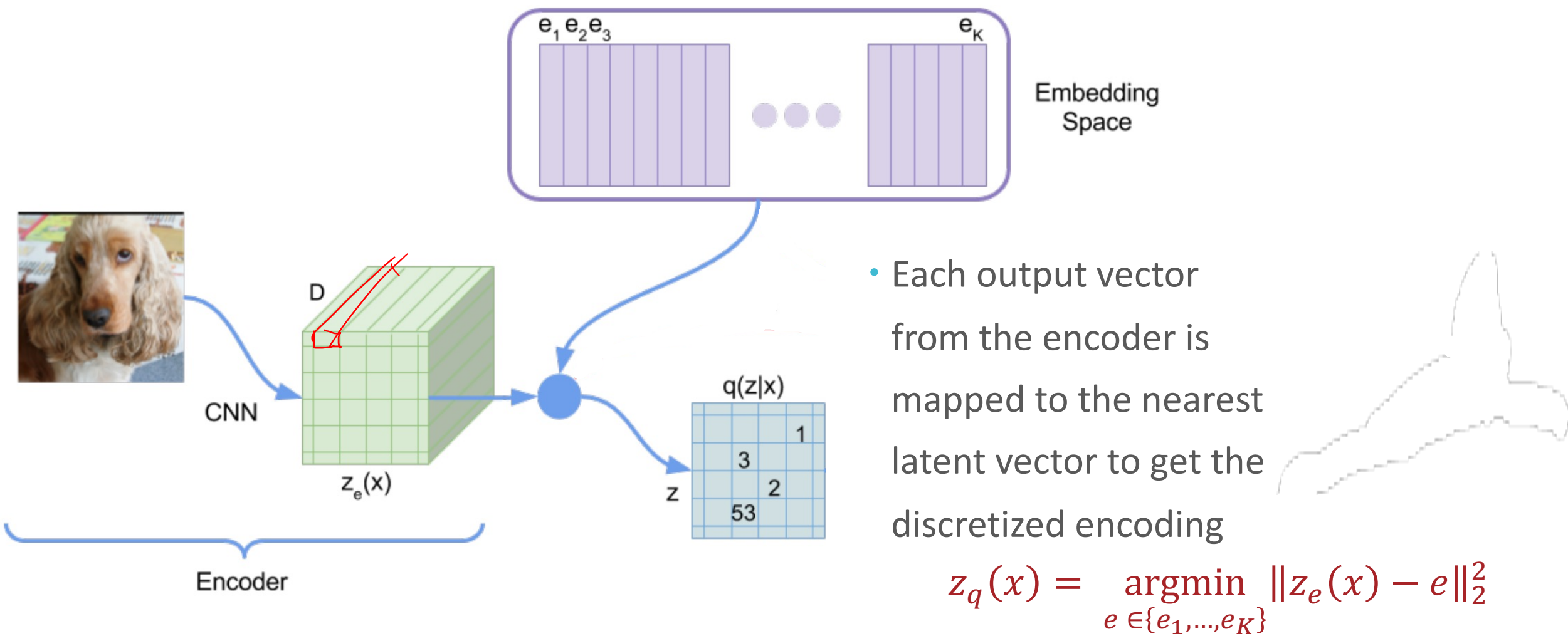


Vector-Quantized VAEs

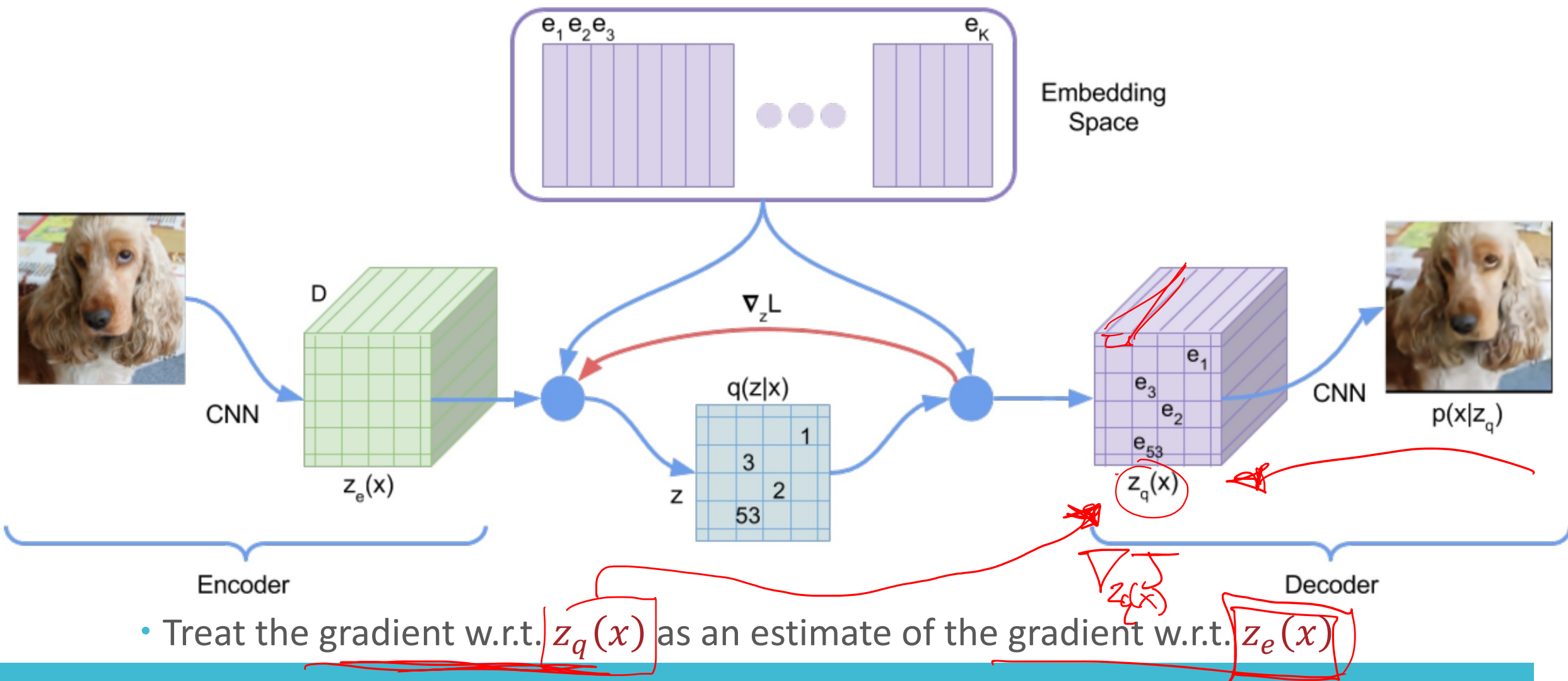


- The decoder takes the discretized representation and recreates the original image

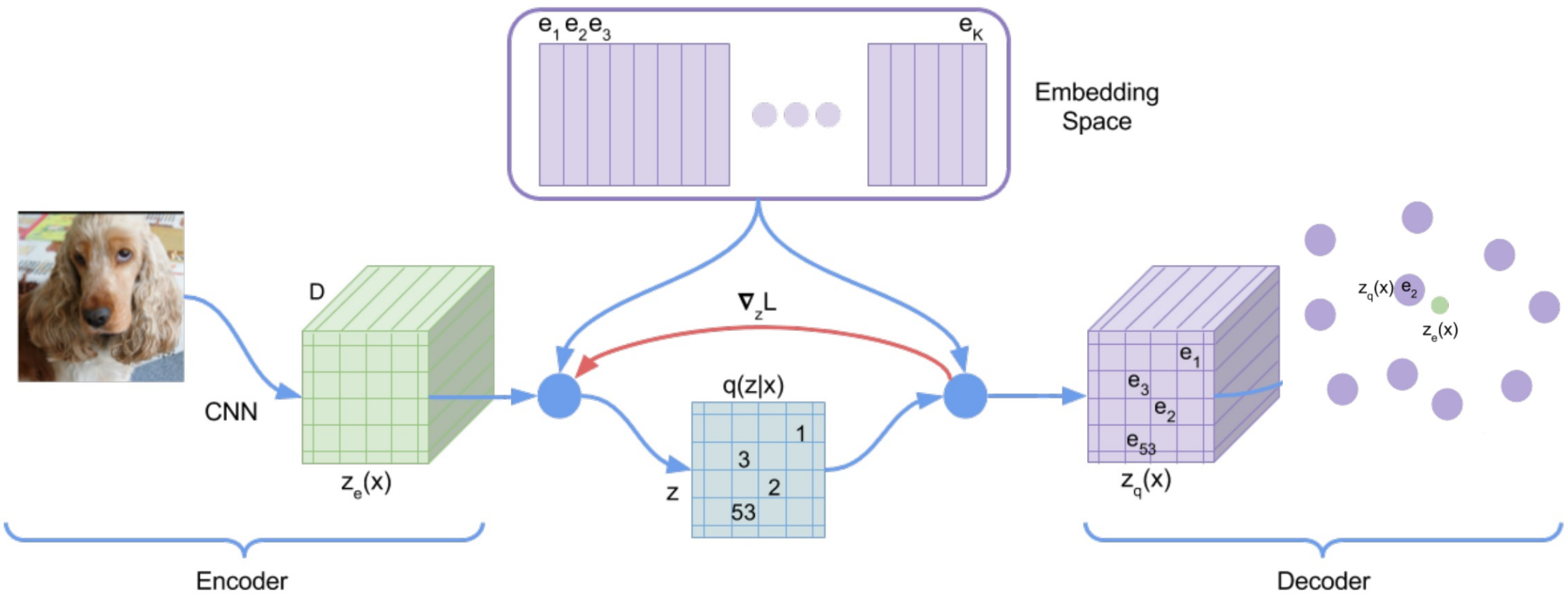
Vector-Quantized VAEs



Wait, how would we take the gradient through the argmin?



Straight-through Estimator



- Intuition: the closer $z_q(x)$ and $z_e(x)$, the better the estimate (under certain assumptions)

Straight-through Estimator

VQ-VAE Objective Function

- Intuition: we want the latent vectors to correspond to relevant points in the embedding space i.e., ones that are near the outputs of the encoder
- However, we also want the encoder to respect the latent vectors and not overfit to the training dataset
- Idea: augment the standard VAE objective with some regularizing terms that drive the two closer to each other

$$\underbrace{-\log p_{\theta}(x|z_q(x))}_{\text{VAE Objective}} + \underbrace{\left\| \text{sg}[z_e(x)] - z_q(x) \right\|_2^2}_{\text{Regularizing Term}} + \underbrace{\beta \left\| z_e(x) - \text{sg}[z_q(x)] \right\|_2^2}_{\text{Regularizing Term}}$$

where **sg** is the stop-gradient operator which fixes the argument to be non-updated constant

VQ-VAE Objective Function

- Intuition: we want the latent vectors to correspond to relevant points in the embedding space i.e., ones that are near the outputs of the encoder
- However, we also want the encoder to respect the latent vectors and not overfit to the training dataset
- Idea: augment the standard VAE objective with some regularizing terms that drive the two closer to each other

$$-\log p_{\theta}(x|z_q(x)) + \left\| \text{sg}[z_e(x)] - z_q(x) \right\|_2^2 + \beta \left\| z_e(x) - \text{sg}[z_q(x)] \right\|_2^2$$

- The first term is the typical reconstruction error objective

VQ-VAE Objective Function

- Intuition: we want the latent vectors to correspond to relevant points in the embedding space i.e., ones that are near the outputs of the encoder
- However, we also want the encoder to respect the latent vectors and not overfit to the training dataset
- Idea: augment the standard VAE objective with some regularizing terms that drive the two closer to each other

$$-\log p_{\theta}(x|z_q(x)) + \left\| \text{sg}[z_e(x)] - z_q(x) \right\|_2^2 + \beta \left\| z_e(x) - \text{sg}[z_q(x)] \right\|_2^2$$

- The second term drives the latent vector to be closer to the encoder output vector that was mapped to it

VQ-VAE Objective Function

- Intuition: we want the latent vectors to correspond to relevant points in the embedding space i.e., ones that are near the outputs of the encoder
- However, we also want the encoder to respect the latent vectors and not overfit to the training dataset
- Idea: augment the standard VAE objective with some regularizing terms that drive the two closer to each other

$$-\log p_{\theta}(x|z_q(x)) + \|sg[z_e(x)] - z_q(x)\|_2^2 \\ + \beta \|z_e(x) - sg[z_q(x)]\|_2^2$$

- The third term drives the encoder to output vectors closer to the latent vectors

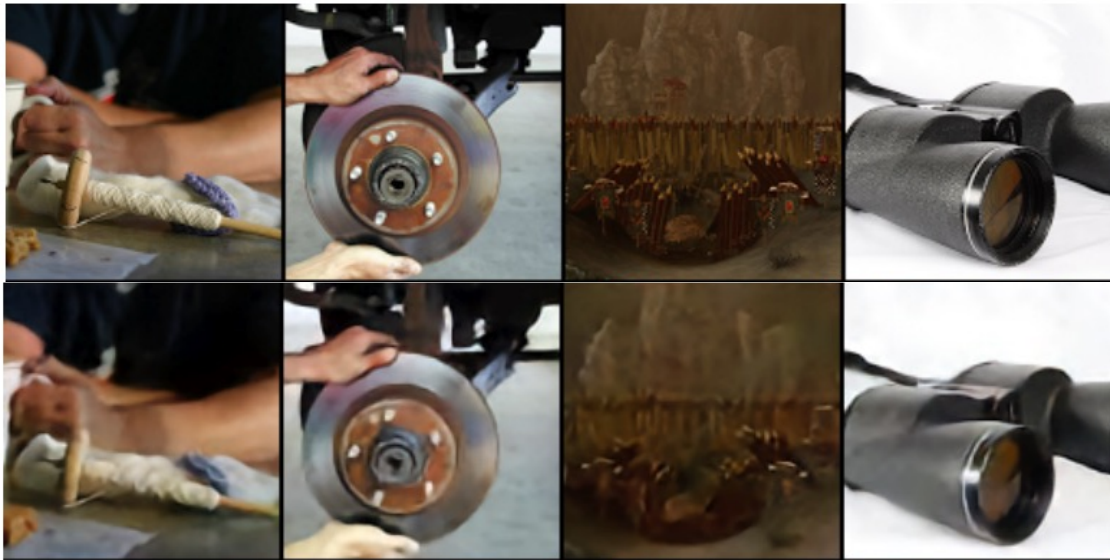
CLIP vs. VQ-VAEs

- VLMs with VQ-VAE encoders (or any vector quantized image model) can also generate images in addition to text by defining a loss over the image codebook tokens
- CLIP does not discretize its image embedding so VLMs with CLIP-based encoders cannot (naturally) define a loss over images and thus, can only output text
- However, CLIP embeddings are more expressive than the discrete VQ-VAE encodings so can lead to improved performance in some settings

VQ-VAE Results

- $p(x|z)$ is a PixelCNN
- input images are 128x128x3 from ImageNet
- compressed to 32x32x1 (K=512)
- reconstructions (bottom) are of quality near the originals (top)
- Obtained a reduction of 42.6 in bits

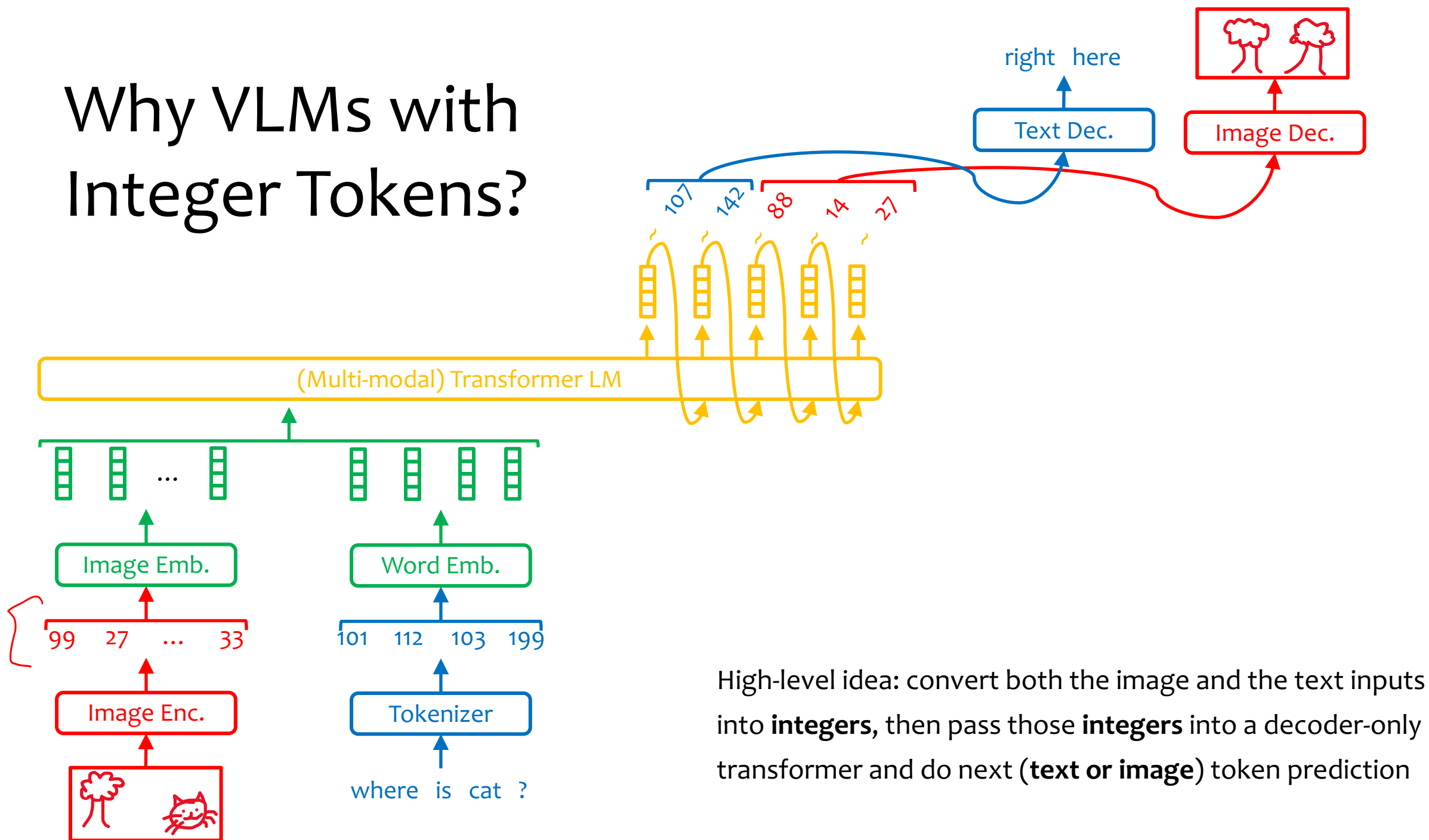
$$\frac{128 \times 128 \times 3 \times 8}{32 \times 32 \times 9} \approx 42.6$$



(VLMs that read text/images and write text/images)

VLMS WITH TEXT AND IMAGE DECODERS

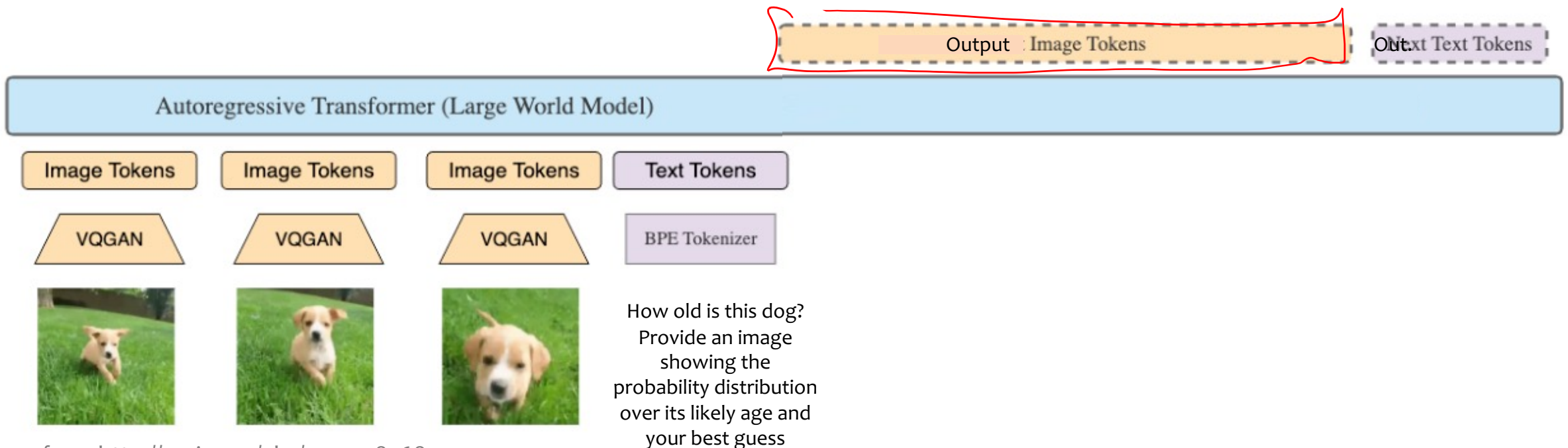
Why VLMs with Integer Tokens?



High-level idea: convert both the image and the text inputs into **integers**, then pass those **integers** into a decoder-only transformer and do next (**text or image**) token prediction

Large World Model

- The Large World Model (LWM) is an example of a Transformer LM that works with both discrete text tokens and discrete image tokens
- Key idea:
 - pretrain your image tokenizer/de-tokenizer (VQGAN)
 - any images in your data can be converted to their discrete representations ahead of time
 - then train your Transformer LM as you would any other LM on discrete tokens
 - at test time, whenever you see a sequence of image tokens, convert them back to an image



Large World Model

H MORE IMAGE GENERATION EXAMPLES



A black dog



A blue colored pizza



A cube made of denim



A glass of wine



*A yellow and black bus
cruising through a rainforest*



*Oil painting of a couple in
formal attire caught in the
rain without umbrellas*



*A couch in a cozy living
room*



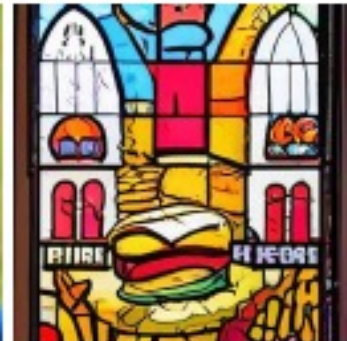
*A carrot to the left of
broccoli*



*Fisheye lens of a turtle
in a forest*



A blue colored dog

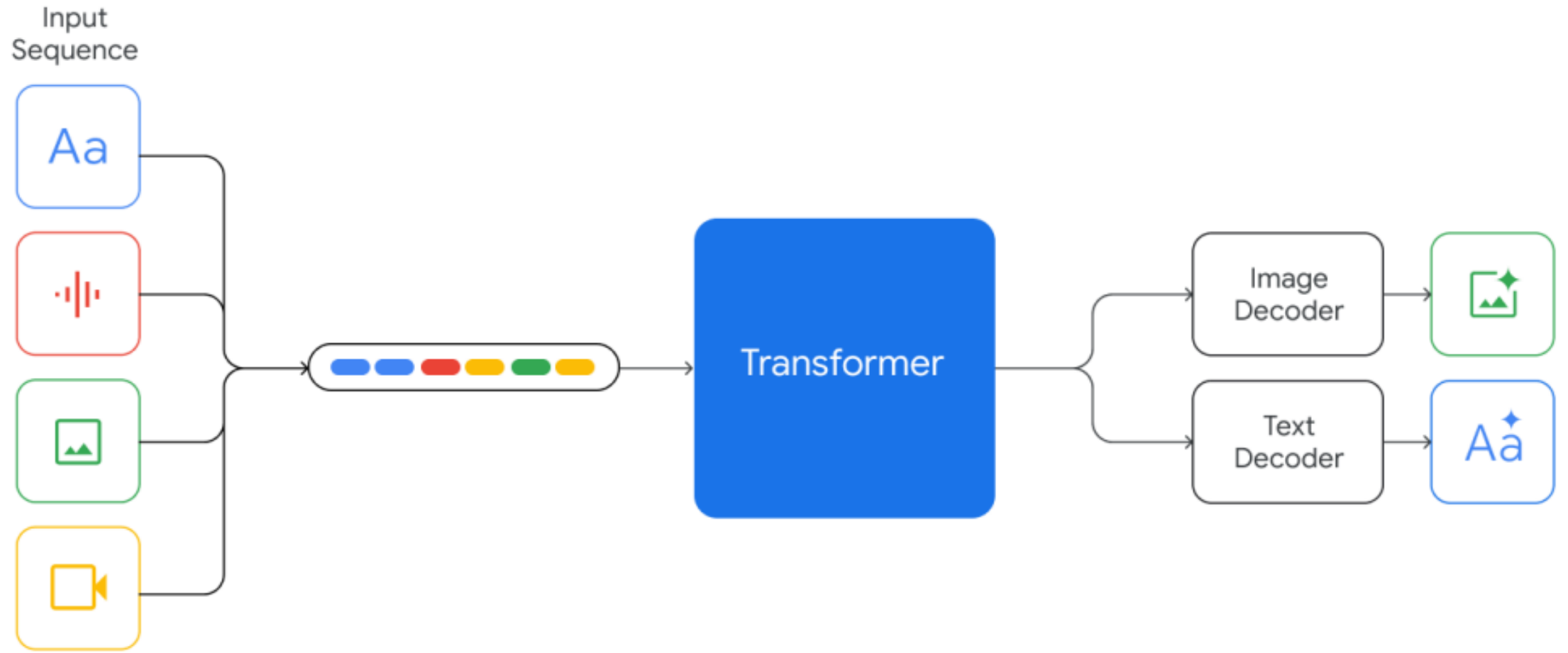


*Stained glass windows
depicting hamburgers and
french fries*



A pink car

Gemini



Gemini

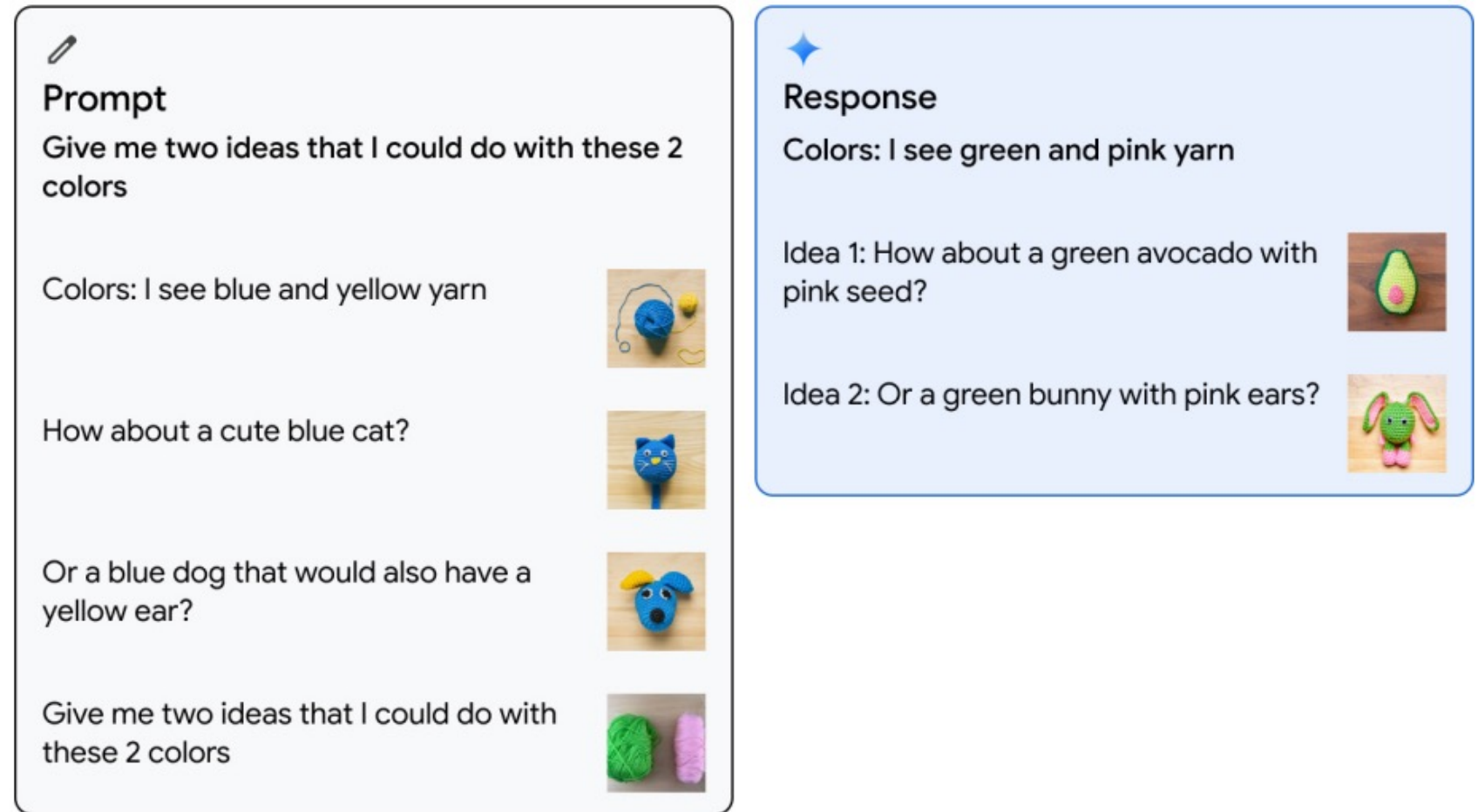
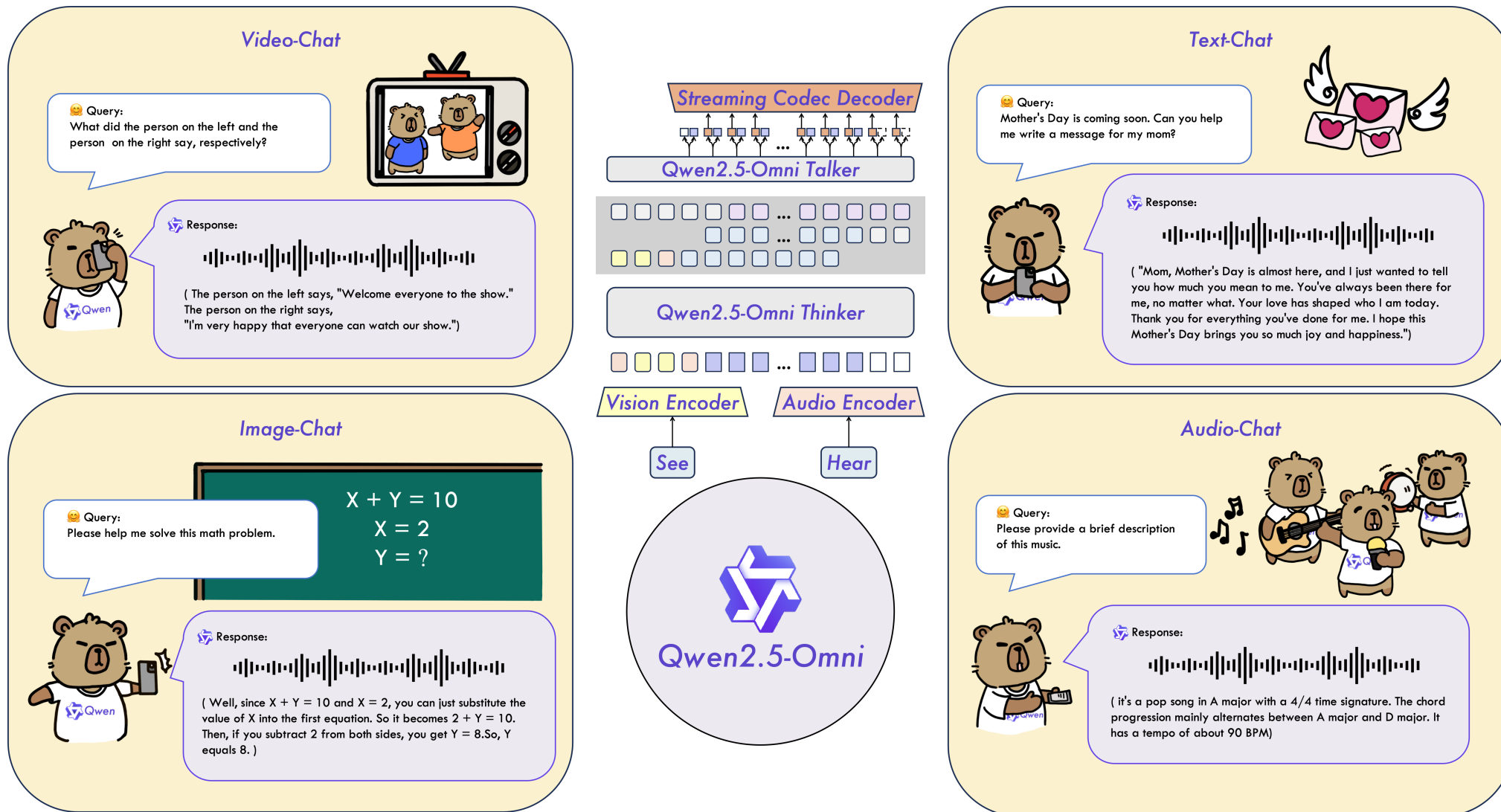


Figure 6 | **Image Generation.** Gemini models can output multiple images interleaved with text given a prompt composed of image and text. In the left figure, Gemini Ultra is prompted in a 1-shot setting with a user example of generating suggestions of creating cat and dog from yarn when given two colors, blue and yellow. Then, the model is prompted to generate creative suggestions with two new colors, pink and green, and it generates images of creative suggestions to make a cute green avocado with pink seed or a green bunny with pink ears from yarn as shown in the right figure.

Any-to-any VLMs



Any-to-any VLMs

• Input Modalities:

- Text: Supports 119 languages.
- Image: Improved visual reasoning and OCR.
- Audio: Processes recordings over 40 minutes long; supports speech input in 19 languages.
- Video: Enhanced audio-visual synchronization (lip-syncing/scene reasoning).

• Output Modalities:

- Text: Includes a “Thinking” mode for chain-of-thought reasoning.
- Audio (Speech): Higher-quality synthesis in 10 languages

